Best Practices for Using Claude

From Personal Mastery to Enterprise Scale

What Anthropic Recommends, What Practitioners Discover, and Where They Converge

Brandon Behring

Version 2.3
February 2026

Contents

Preamble

This handbook teaches you to use Claude Code effectively—not by listing features, but by walking through the workflows where each feature earns its value.

Two Sources of Knowledge

Every practice is drawn from one of two sources and labeled accordingly:

[Official]
: Anthropic's documented recommendation, verified against current documentation with source URL. Example: **[Official]** Run `/init` to generate a starter CLAUDE.md.[1]

[Practitioner]
: Discovered through extensive daily use across 20+ projects spanning data science, ML, and software engineering. Example: **[Practitioner]** After two failed corrections, `/clear` and write a better initial prompt.

[Convergence]
: Both sources agree independently. Example: **[Convergence]** Verification criteria in CLAUDE.md from day one.

How This Book Is Organized

The handbook follows your adoption journey through four parts, each corresponding to a level of the Claude Maturity Model (Chapter A):

Part I: Foundations
: (Part I) — What Claude Code is, your first CLAUDE.md, your first session. *Takes you from L1 (Conversational) to L2 (Configured).*

Part II: Personal Practice
: (Part II) — Advanced prompting, context management, testing, extensions. *Takes you from L2 to L3 (Systematic).*

Part III: Advanced Craft
: (Part III) — Configuration architecture, agents, project lifecycle, anti-patterns. *Takes you from L3 toward L4 (Autonomous).*

Part IV: Team & Enterprise
: (Part IV) — Automation, team governance, enterprise deployment. *Scaling from L4 to L5 (Enterprise).*

Who This Is For

Primary audience: Data scientists and ML engineers who have used Claude Code at least once and want to go from "it works sometimes" to "it works reliably." No prior configuration experience assumed.

Secondary audience: DS/ML team leads evaluating Claude for their teams. Chapter 15 is specifically designed to be shared with decision-makers.

[1] https://code.claude.com/docs/en/best-practices

How to Read

New to Claude Code?
Start at Chapter 1 and read straight through Part I.

Already configured?
Start at Chapter 4 (Part II). Skim Part I for anything you missed.

Experienced user?
Jump to Chapter 12. Fix what is broken. Then read Chapter 9 for scaling patterns.

DS/ML team lead?
Read Chapter 15 and Chapter A, plus Chapter 5 and Chapter 7 for DS-specific workflow value.

Spiral Structure

Key concepts appear at increasing depth as you progress:

- **CLAUDE.md**: basics in Chapter 2, architecture in Chapter 9
- **Context**: basics in Chapter 3, full management in Chapter 6
- **Testing**: overview in Chapter 7, characterization tests in Chapter 11
- **Extensions**: overview in Chapter 8, agents and delegation in Chapter 10
- **Collaboration**: thinking partner in Chapter 5, writer/reviewer in Chapter 10

Each revisit explicitly connects back to the earlier treatment.

A Note on Pace

AI tooling moves fast. Content is verified as of February 2026. Check the repository for updates, and consult Chapter D for primary source URLs.

Part I.

Foundations

You are at Level 1 (Conversational)—using Claude Code as a chatbot with file access. By the end of this part, you will be at Level 2 (Configured): Claude will know your project, respect your boundaries, and follow your conventions without being told twice.

1. Principles and Mental Model

You have just installed Claude Code. Before your first prompt, take five minutes to understand what you are working with—not a chatbot, not an autocomplete engine, but an agent with a specific architecture and specific constraints. The principles in this chapter will inform every decision in this handbook.

1.1. What Claude Code Actually Is

Claude Code runs an **agent loop**: it reads your prompt, reasons about the task, selects a tool (read a file, edit code, run a command), observes the result, and decides what to do next. This cycle repeats until the task is complete or Claude asks for clarification.

Official
Claude Code is an agentic coding tool that operates directly in your terminal.
https://code.claude.com/docs/en/overview

Three properties define the system:

Context window
: Everything Claude knows about your task—your prompt, files it has read, tool results, conversation history—occupies a finite window. When the window fills, older information fades. Context is not infinite memory; it is a workspace.

Tool use
: Claude does not type code into a file. It calls tools: `Read`, `Edit`, `Write`, `Bash`, `Glob`, `Grep`. Each tool call consumes context (the request and the result) and produces observable effects. Understanding this matters because every tool call has a cost in context tokens.

Configuration layers
: Claude's behavior is shaped by four layers: `CLAUDE.md` (project knowledge), rules (conditional context), settings (permissions and behavior), and output styles (communication patterns). These layers are not suggestions—they are the operating system through which Claude interprets your project.

Why the Mental Model Matters
Developers who treat Claude as a chatbot prompt it like one: vague requests, no verification, no context management. Developers who understand the agent loop prompt it like a system: clear inputs, verification criteria, deliberate context control. The same tool produces dramatically different results depending on which mental model the developer holds.

1.2. Four Engineering Principles

These four principles underlie every recommendation in this handbook. They apply to code written by AI, code written by humans, and code written by both together.

1.2.1. Never Fail Silently

Every error must be explicitly reported with recovery options. Silent failures are the most expensive kind of bug: they produce incorrect results

that look correct, propagate through downstream systems undetected, and surface only when the damage is difficult to reverse.

Before & After

Before:

```
# CLAUDE.md
Handle errors gracefully.
```

After:

```
# CLAUDE.md
If you encounter an error, report it immediately
with the error message, your analysis of the root cause,
and 2-3 options for resolution.
Never work around errors silently.
```

In AI-assisted development, this principle has an additional dimension: Claude must be configured to surface problems rather than work around them. A vague instruction to "handle errors" invites Claude to catch and suppress. An explicit instruction to "report errors with analysis" makes every failure visible and actionable.

Tip
The "after" version is longer but produces better outcomes. Precision in error handling instructions compounds across every session.

1.2.2. Simplicity Over Complexity

Short functions. Flat structure. Self-documenting code. A 20-line function with a clear name is superior to a 5-line function with three levels of abstraction.

Why Simplicity Matters More with AI

Simple code is easier for Claude to reason about, test, and modify correctly. Complex abstractions increase the probability of subtle bugs in AI-generated code because each layer of indirection is another opportunity for misunderstanding. When Claude reads your codebase to learn conventions, simple patterns propagate accurately; complex patterns propagate errors.

Tip
In notebooks, this means: one cell, one purpose. Cells that do loading, cleaning, AND analysis violate this principle.

1.2.3. Immutability by Default

Return new data structures; mark mutations explicitly. Pure functions are easier to test, easier to parallelize, and easier for Claude to reason about.

When mutation is necessary (performance, I/O, state management), make it explicit: name the function `update_`, use mutable types, and document the side effects. The reader—human or AI—should never be surprised by what a function modifies.

1.2.4. Fail Fast with Diagnostics

Stop immediately on problems with full context: what failed, what was expected, what was received, and what the caller can do about it.

```
def process_data(
    df: pd.DataFrame, min_rows: int
) -> pd.DataFrame:
```

```
    if len(df) < min_rows:
        raise ValueError(
            f"Need {min_rows} rows, got {len(df)}. "
            f"Check data source or reduce "
            f"min_rows parameter."
        )
    # ... process
```

The error message includes: what went wrong (`Need N, got M`), where to look (`data source`), and what to do about it (`reduce min_rows`). This is the difference between a 10-second fix and a 30-minute investigation.

Tip
Error messages that include both the violation and a suggested fix reduce debugging time dramatically.

1.3. The Codebase Is the Curriculum

Key Insight
T

hese principles are not Claude-specific—they are engineering principles that become *more* important when AI is generating code. AI amplifies both good and bad patterns. If your codebase follows these principles, Claude's contributions will follow them too. If your codebase is silent about errors, full of mutable state, and deeply abstracted, Claude's contributions will be the same. The codebase is the curriculum. Make it teach the right lessons. For data scientists, this includes your notebooks, pipeline scripts, and feature engineering code—Claude learns patterns from all of them.

This insight has a practical consequence: the single most valuable thing you can do before using Claude on an existing project is to ensure that the code Claude will read exemplifies the patterns you want Claude to produce. Configuration (Chapter 2) reinforces this, but the codebase itself is the primary teacher.

Cross-Ref
For concrete techniques to make your codebase a better teacher, see Chapter 5.

1.4. Quick Reference

- Claude Code is an agent loop: prompt → reason → tool → observe → repeat
- Context window is finite—every tool call costs tokens
- Four principles: never fail silently, simplicity, immutability, fail fast
- The codebase teaches Claude patterns—good and bad
- Configuration (next chapter) reinforces but does not replace good code

Try This
Pick one function in your feature pipeline or data loading code—ideally one Claude will read often. Audit it against the four principles:

1. Does it fail silently anywhere? (e.g., silently dropping NaN rows without logging)
2. Is it under 30 lines? If not, can you extract the transformation logic from the I/O?
3. Does it mutate its input DataFrame? If so, is that explicit in the name?
4. Do its error messages include what went wrong AND what to do about it?

> Fix one issue. This is your first investment in the codebase-as-curriculum.

2. Your First CLAUDE.md

Claude keeps asking you the same questions about your project. It does not know your build command, your test runner, or your architecture. Every session starts with you re-explaining context that should be obvious. The fix takes five minutes.

2.1. What CLAUDE.md Does

[Official] The `CLAUDE.md` file is the single most important configuration artifact.[1] Claude reads it at session start and uses it to understand your project's conventions, build commands, architecture decisions, and rules of engagement.

Official
Run `/init` to auto-generate a starter CLAUDE.md from your project structure.
https://code.claude.com/docs/en/best-practices

Think of it as a briefing document for a new team member who is fast, capable, and amnesiac—they forget everything between sessions unless you write it down.

2.1.1. Hierarchical Loading

Claude loads `CLAUDE.md` files from multiple locations, merging them in order:

1. `~/.claude/CLAUDE.md` — global defaults (applies to all projects)
2. Project root `CLAUDE.md` or `.claude/CLAUDE.md` — project-level instructions
3. Parent and child directory `CLAUDE.md` files — subdirectory overrides

For now, you only need the project-level file. The global file and advanced architecture come later (Chapter 9).

2.2. Generate It: `/init`

The fastest path to a useful CLAUDE.md:

1. Open your project in Claude Code.
2. Type `/init`.
3. Claude examines your project and proposes a starter CLAUDE.md.
4. Review, edit, commit.

The auto-generated version is a solid starting point. What follows are the refinements that make it effective.

2.3. What to Include

[Convergence] Include what Claude cannot infer. Exclude what it can.

Build and test commands
: `pytest tests/`, `mypy src/`, `ruff check src/`—anything non-standard or project-specific.

Code style rules
: Only those that differ from standard conventions. Do not restate PEP 8 defaults.

[1] https://code.claude.com/docs/en/best-practices

Architecture decisions
: Where things go and why. "Feature pipelines in `src/features/`. Models in `src/models/`. Notebooks in `notebooks/`."

Verification criteria
: The single highest-leverage inclusion. "Always run tests after code changes. Never commit without passing CI."

Before & After

Before:

```
# My Project
This is an ML project. We use scikit-learn and pandas.
Tests are important to us.
```

After:

```
# Demand Forecasting

## Build & Verify
- `pytest tests/` — run test suite
- `mypy src/` — type checking
- `ruff check src/` — lint

## Architecture
- src/features/ — feature engineering pipelines
- src/models/ — model training and evaluation
- src/data/ — data loading and validation
- notebooks/ — exploratory analysis
- data/raw/ — immutable source data
- data/processed/ — transformed artifacts
- tests/ — mirrors src/ structure

## Standards
- Type hints on all function signatures
- Docstrings on all public functions
- Run tests after every code change
- Never commit without passing lint + tests
```

2.4. What to Exclude

- What Claude can infer from the codebase (standard language conventions)
- Long explanations—keep instructions terse and actionable
- Aspirational rules nobody follows—document what IS, not what you wish
- Information that changes frequently (put it in memory instead—see Chapter 6)

Tip
The "before" tells Claude about your project. The "after" tells Claude how to *work* on your project. Commands, locations, and rules—not descriptions.

Quality Test
If Claude keeps violating a rule, one of three things is true: (1) the CLAUDE.md is too long and the rule is getting lost, (2) the rule contradicts another rule, or (3) the rule should be a hook instead of an instruction (see Chapter 8).

Write Positive Instructions

Replace negative constraints with positive alternatives. *"Never use raw SQL"* leaves Claude uncertain about what to use. *"Always use the ORM query builder in `src/db/queries.py`"* tells Claude exactly what to do instead. Positive instructions are clearer, more actionable, and more reliably followed.

2.5. Your First Deny Rules

Before Claude touches any code, protect your secrets. Create `.claude/settings.json`:

Official

Settings documentation:
https://code.claude.com/docs/en/settings

```
{
  "permissions": {
    "deny": [
      "Read(.env*)",
      "Read(secrets/**)",
      "Read(**/*.pem)",
      "Read(**/*.key)",
      "Read(**/credentials*)"
    ]
  }
}
```

Deny rules are the one configuration that is never wrong to add on day one. They cost nothing in context, they never cause false negatives in normal work, and they prevent the one class of mistake that is truly irreversible: leaking secrets into a commit, a log, or a conversation.

Tip

Deny rules are absolute. Unlike CLAUDE.md instructions (which are advisory), deny rules cannot be overridden by conversation or prompt injection.

2.6. Quick Reference

- Run `/init` to generate a starter CLAUDE.md
- Include: build commands, architecture, code style, verification criteria
- Exclude: what Claude can infer, long prose, aspirational rules
- Add deny rules for secrets in `.claude/settings.json` on day one
- Commit both files to version control

Try This

1. Run `/init` on your current project.
2. Compare the generated CLAUDE.md to what you actually need. Add `pytest`, `mypy`, and `ruff` commands if missing.
3. Create `.claude/settings.json` with deny rules for `.env*`, `data/raw/**`, and any credentials directories.
4. Start a new session and verify Claude reads your CLAUDE.md (ask: "What are my build commands?").

Time: ~5 minutes.

3. Your First Working Session

Your CLAUDE.md is set up and your secrets are protected. You open Claude Code, type a prompt, and…where do you start? How do you structure a request so Claude does what you mean? How do you know when to start fresh versus keep going?

By the end of this chapter, you will have:

- A repeatable session loop: prompt → observe → refine → commit
- Three prompting techniques that handle 80% of interactions
- A context hygiene discipline (two commands, one rule)
- Escape routes for when things go wrong

3.1. The Session Loop

A productive session with Claude follows a repeating cycle:

The Session Loop

1. **Prompt** — describe what you want, with enough context for Claude to act without guessing.
2. **Observe** — read what Claude did. Check the diff, run the tests, examine the output.
3. **Refine** — if the result is close but not right, give specific feedback. If it is wrong, start fresh (`/clear`).
4. **Commit** — when the change is verified, commit it. One logical change per commit.

The rest of this chapter teaches three essential skills for the prompt and refine steps: how to write clear requests, how to provide useful feedback, and how to manage the conversation so it stays productive.

Cross-Ref

If Claude goes off track: `Esc` stops mid-action, `Esc+Esc` opens the rewind menu. Full details in Section 3.4.

3.2. Three Prompting Basics

These three techniques handle 80% of daily interactions. Chapter 4 covers advanced prompting in depth; this section gives you enough to be productive today.

3.2.1. Be Specific About What, Not How

Before & After

Before:

```
Fix the broken pipeline.
```

After:

```
The feature engineering pipeline in
src/features/temporal.py raises a KeyError on line 34.
The error is: KeyError: 'timestamp'.
The column exists in training data but is missing
from the inference schema. Diagnose and fix.
```

The "before" makes Claude guess which tests, what kind of failure, and what "fix" means. The "after" gives Claude the information it needs to act precisely.

3.2.2. Tell Claude to Think Step by Step

For complex tasks, give Claude an explicit reasoning structure:

Official
Step-by-step prompting is consistently identified as one of the three highest-leverage techniques.
https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview

```
Refactor the feature engineering pipeline:
1. Read the current implementation and
   list all transformation steps.
2. Identify steps that mix data loading
   with transformation.
3. Extract pure transformation functions.
4. Run existing tests after each extraction
   to verify nothing broke.
```

Without this structure, Claude may attempt the entire refactoring in one pass, increasing the risk of subtle regressions.

3.2.3. Include Verification Criteria

[Convergence] "Give Claude a way to verify its own work" is Anthropic's stated single highest-leverage practice.[1]

Always include how to check the result:

```
Add input validation to the build_features() function.
Verify: pytest tests/features/test_build.py passes,
and test with:
python -c 'from src.features import build_features;
  build_features(pd.DataFrame())'
(should raise ValueError, not silently return empty).
```

When Claude knows the success criteria, it can self-correct before you ever see the result.

From Typing to Defining

You just learned three techniques: specificity, step-by-step reasoning, and verification criteria. Notice the shift—none of them are about writing code. They are about defining what "correct" means precisely enough that Claude can produce it. Your value moved from implementation to specification. The rest of this chapter protects that investment: keeping context clean so these techniques stay effective.

3.3. Context Hygiene: The Basics

The context window (introduced in Section 1.1) is your workspace. Like a physical desk, it becomes less effective as it fills with unrelated materials.

Official
Use `/clear` between unrelated tasks. Use `/compact` for selective summarization.
https://code.claude.com/docs/en/best-practices

3.3.1. Two Commands to Know Now

`/clear`
: Wipe context and start fresh. Use between unrelated tasks: after fixing a bug, before writing a feature.

[1] https://code.claude.com/docs/en/best-practices

/compact
: Summarize the conversation while preserving key decisions. Less aggressive than `/clear`; use when you need to continue a long task but context is getting large.

3.3.2. The Two-Failure Rule

[Practitioner] After two failed corrections on the same issue, `/clear` and write a better initial prompt.

Each correction attempt adds noise: the original error, your correction, Claude's apology, and its retry. After two rounds, the context is dominated by failure patterns rather than the actual task. Starting fresh with a better prompt costs 30 seconds and produces better results than a third correction round.

When to Clear
If you find yourself typing "no, that's not what I meant" for the third time, stop. `/clear`. Write the prompt you wish you had written the first time. The cost of accumulated correction context exceeds the cost of starting fresh.

3.4. Course-Correcting

[Convergence] The best results come from tight feedback loops. Correct Claude as soon as you notice it going off track.[2]

Esc
: Stop Claude mid-action. Context is preserved, so you can redirect immediately.

Esc + Esc
: Open the rewind menu: restore conversation, code, or both to any previous checkpoint.

``Undo that''
: Have Claude revert its changes.

/clear
: Reset context entirely between unrelated tasks.

Every action Claude takes creates a checkpoint automatically. Think of checkpoints as free save points—you can always go back if Claude takes a wrong turn.

Why This Matters
You now have the tools to recover from any wrong turn. The next section adds one more capability: letting Claude explore and plan *before* it builds, so wrong turns happen less often.

Tip
If you have corrected Claude more than twice on the same issue, the context is cluttered with failed approaches. `/clear` and start fresh with a better prompt.

3.5. Plan Mode: Explore Before You Build

For complex or unfamiliar tasks, let Claude understand the codebase before making changes. Press `Shift+Tab` twice to enter Plan Mode—Claude can read files and answer questions but cannot modify anything. Use `Ctrl+G` to open the plan in an external text editor for detailed review, or start directly

Official
Separate research from implementation to avoid solving the wrong problem.
https://code.claude.com/docs/en/best-practices

[2] https://code.claude.com/docs/en/best-practices

in Plan Mode from the command line with `claude --permission-mode plan`.

[Official] The recommended workflow for complex tasks has four phases:[3]

1. **Explore** (Plan Mode): "Read `src/features/` and explain how we compute temporal features."
2. **Plan** (Plan Mode): "I want to add recency features. What files need to change? Create a plan."
3. **Implement** (Default Mode): "Implement the plan. Write tests first, then code. Run the test suite."
4. **Commit**: "Commit with a descriptive message."

Official
Five permission modes exist: Default, Accept Edits, Plan (keyboard-cyclable via `Shift+Tab`), plus `dontAsk` and `bypassPermissions` (settings-only).
https://code.claude.com/docs/en/permissions

When to Skip Planning

Planning adds overhead. Skip it when the scope is clear and the fix is small (typo, log line, variable rename). If you could describe the diff in one sentence, just ask Claude to do it directly. Plan when you are uncertain about the approach, the change spans multiple files, or you are unfamiliar with the code.

3.6. A Complete Example

Here is a realistic mini-session showing the loop in practice:

```
You: Add missing value imputation to the preprocessing
     pipeline. Strategy: median for numeric columns,
     mode for categorical. Write tests first.
     Verify: pytest tests/preprocessing/ passes.

Claude: [writes tests, then implementation, runs tests — 3 pass, 1 fails]
  "Test test_categorical_imputation is failing because..."

You: Mode should use the training set distribution,
     not the test batch. Fix the imputer to fit on train.

Claude: [fixes implementation, reruns — all pass]
  "All 4 tests pass. Ready to commit?"

You: Yes, commit with message "feat: Add missing value imputation"
```

Notice the structure:

- The initial prompt included what, how (strategy), and verification criteria.
- The refinement was specific (which behavior, what to change).
- One logical change, one commit.

3.7. Quick Reference

- Session loop: prompt → observe → refine → commit
- Be specific about *what* you want, not *how* to do it
- Include verification criteria in every prompt
- Plan Mode (Shift+Tab × 2) for complex or unfamiliar tasks
- `Esc` to stop mid-action, `Esc+Esc` to rewind
- Use `/clear` between unrelated tasks

[3] https://code.claude.com/docs/en/best-practices

- Two-failure rule: after two corrections, start fresh

What you practiced:

- Specification: writing prompts that include what, context, and verification criteria
- Observation: reading diffs and test results before approving
- Context discipline: clearing between tasks, compacting when long
- Course-correction: stopping, rewinding, starting fresh

Try This

Start a session with Claude Code on your current project. Add one feature to your data pipeline or fix one data processing bug:

1. Write a prompt that includes: what you want, the relevant DataFrame schema, and how to verify (e.g., `pytest tests/features/`).
2. Let Claude work. Observe the result.
3. If refinement is needed, give specific feedback (file, line, which column or transformation to change).
4. When satisfied, commit the change.

Track how many refinement rounds it takes. If it exceeds two, try `/clear` and a better initial prompt.

Part II.

Personal Practice

You are at Level 2 (Configured)—Claude knows your project and follows basic rules. This part takes you to Level 3 (Systematic): you will manage context deliberately, test AI-generated code rigorously, and extend Claude with custom capabilities that encode your team's workflows.

4. Prompting That Works

Your prompts work, but they are inconsistent. Sometimes Claude nails it on the first try; sometimes you spend three rounds correcting misunderstandings. The gap is not randomness—it is precision.

In Chapter 3, you learned three prompting basics: be specific, think step by step, include verification criteria. This chapter goes deeper: building a precision vocabulary, structuring complex prompts, managing cost, and using extended thinking.

4.1. Building a Precision Vocabulary

[Practitioner] Track how your requests could be stated more precisely. Over time, develop a shared vocabulary between you and Claude.

Natural Language	Precise Version
"Clean this data"	`validate schema, impute nulls with median, drop rows where target is NaN`
"Train a model"	`fit XGBoost on train split, evaluate AUC on val, log params to MLflow`
"Check if this works"	`validate: run pytest, check no data leakage, verify feature distributions match prod`
"Add error handling"	`add validation to build_features(): raise ValueError on schema mismatch, log and skip rows with >50% NaN`

Why Precision Vocabulary Compounds

Precision is not just about saving correction rounds in a single session. It builds a shared language between you and Claude. When your CLAUDE.md says "validate schema" and your prompts say "validate schema," Claude interprets both consistently. This alignment compounds: your prompts become shorter, your CLAUDE.md rules more effective, and your correction rate drops—not linearly, but exponentially as the vocabulary stabilizes.

Tip

Precision compounds. Practitioners report that shared vocabulary can reduce correction rounds by 2–3×—a plausible estimate, though hard to measure precisely and workload-dependent.

The pattern for building vocabulary:

1. After each interaction, note how the request could have been clearer.
2. Identify recurring request patterns (create, diagnose, refactor, validate).
3. Document these patterns in your project nomenclature.
4. Use the precise version in future requests.

4.2. Structuring Complex Prompts

4.2.1. XML Tags for Multi-Component Prompts

Cross-Ref

For using precision prompts in collaborative reasoning, see Chapter 5.

[Official] For complex prompts, use XML tags to separate concerns:[1]

```
<context>
The churn prediction model uses features from
src/features/customer.py. Current AUC is 0.72
on validation.
</context>

<task>
Add recency features: days since last purchase,
days since last login.
Compute from the events table.
</task>

<constraints>
- No data leakage: features must use only
  pre-churn data
- Must work with the existing FeatureStore
  interface
- Include unit tests for the new features
</constraints>

<verification>
Run: pytest tests/features/ -v
All existing tests must still pass.
Verify: no future-dated features in test set.
</verification>
```

4.2.2. Rich Inputs

[Official] Use `@file` to include files, paste screenshots, pipe data from `stdin`.[2] Real data beats verbal descriptions.

Method	When to Use
`@file` or `@dir/`	Load file or directory content before Claude responds. Preferred over describing code.
Image paste	Copy/paste or drag-and-drop screenshots, diagrams, error messages directly into the prompt.
`cat file | claude`	Pipe data from other tools. Useful for logs, command output, or data samples.
URL allowlisting	Use `/permissions` to allow frequently-used doc sites. Claude can then fetch reference material directly.

```
Look at @src/features/customer.py
and @tests/features/test_customer.py.
The feature builder computes 12 features
but has no validation for missing values.
Add validation following existing patterns.
```

Tip

`@` references load content before Claude responds—more efficient than asking Claude to `Read` the file itself, and it ensures the context is available from the first reasoning step.

[1] https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview

[2] https://code.claude.com/docs/en/best-practices

```
# Pipe error logs for diagnosis
tail -100 app.log | claude -p "Diagnose the errors"

# Pipe test output for analysis
pytest tests/ -v 2>&1 | claude -p "Summarize failures"
```

4.3. Extended Thinking

[Official] Extended thinking lets Claude reason through complex problems before responding.[3] On Opus 4.6, thinking is **adaptive**—the model decides how much to think based on problem complexity.

Official
On by default. Toggle: Alt+T. Opus 4.6: adaptive thinking with effort controls.
https://platform.claude.com/docs/en/build-with-claude/extended-thinking

4.3.1. Effort Controls

`low` Minimal thinking. Best for: simple lookups, formatting, boilerplate.

`medium` Moderate analysis. Best for: standard coding tasks, code reviews.

`high` Deep reasoning. Best for: architecture decisions, complex debugging.

`max` Maximum effort. Best for: strategic decisions, novel problems.

[Official] Interleaved thinking[4]: Claude can think between tool calls, enabling nuanced multi-step workflows where each step's reasoning informs the next.

4.3.2. When to Enable vs. Disable

Extended Thinking

Enable when: the task requires multi-step reasoning, architectural decisions, debugging where the root cause is unclear, or code review where you want to see the thought process.

Disable when: the task is simple (formatting, boilerplate), latency matters more than depth, or you are running bulk operations where speed dominates.

Budget note: Thinking tokens add to your bill and consume context budget. In a long session with context already at 50%, heavy thinking can push you past the degradation threshold faster. Monitor `/context` when using extended thinking in long sessions.

Cross-Ref
Extended thinking is especially valuable during collaborative design sessions (Chapter 5).

4.4. Cost Optimization

4.4.1. Prompt Caching

[Official] Prompt caching reduces input costs by 90% for repeated content.[5]

Cost
Thinking tokens are billed as output tokens. Deep thinking on a complex problem can generate thousands of thinking tokens—visible in session

Cost
Prompt caching: up to 90% on cached reads. Batch API: 50% discount. Combined: up to 70%+ savings (workload-dependent).
https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Ephemeral (5-minute TTL)
Automatic for content appearing in multiple requests within a short window.

[3] https://platform.claude.com/docs/en/build-with-claude/extended-thinking
[4] https://platform.claude.com/docs/en/build-with-claude/extended-thinking
[5] https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Persistent (1-hour TTL)
Explicit cache breakpoints for content you know will be reused: tool definitions, system prompts, reference documents.

4.4.2. Batch API

[Official] 50% discount for asynchronous processing.[6] Submit up to 100,000 requests per batch (256 MB max). Most batches complete within 1 hour; results available within 24 hours. Combinable with caching for up to 70%+ total savings (workload-dependent).

Best for: bulk migrations, code review across many files, documentation generation.

4.4.3. Model Selection

Model Class	Strength	Best For
Haiku 4.5	Speed, lowest cost	Simple tasks, high volume, formatting
Sonnet 4.6	Balance of speed and capability	Most coding tasks, daily development
Opus 4.6	Frontier reasoning	Complex architecture, multi-file refactoring

[Practitioner] Default to Sonnet for development work. Escalate to Opus for architecture decisions, complex debugging, and multi-file refactoring. Use Haiku for bulk operations and simple formatting tasks.

Cost
Pricing changes frequently. Check current rates before planning budgets.
https://platform.claude.com/docs/en/about-claude/pricing

4.5. Visual: Cost Optimization Decision Tree

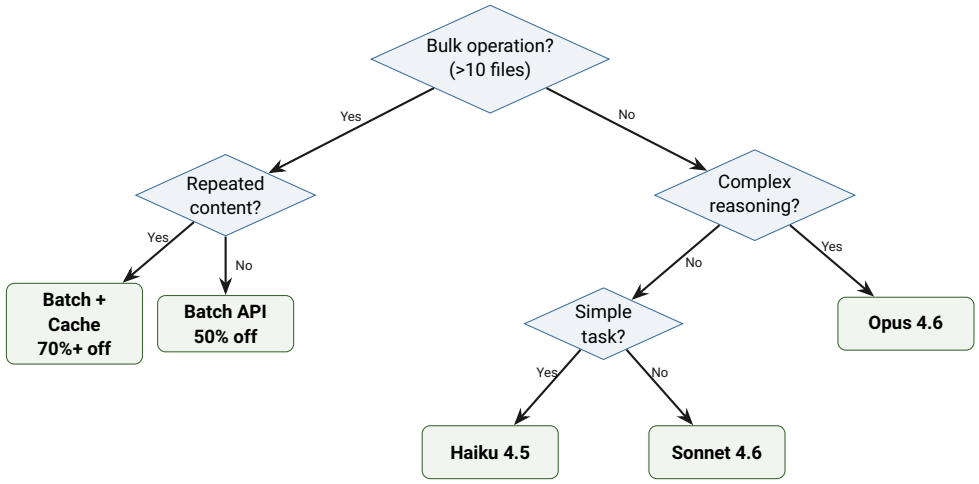

Figure 4.1.: Model and API selection decision tree for cost optimization.

Key Insight
T

he most cost-effective pattern is not choosing the cheapest model—it is structuring your workflow to maximize cache hits. With stable, repeated content, a well-organized CLAUDE.md and tool definitions can reduce per-

[6] https://platform.claude.com/docs/en/build-with-claude/batch-processing

session costs by up to 60–80% through caching.

Try This

Take your last three prompts to Claude (check terminal history). Rewrite each using the precision vocabulary table:

1. Replace vague verbs (“clean,” “train,” “check”) with precise ones (“validate schema and impute nulls,” “fit on train split and evaluate AUC,” “run pytest and verify no leakage”).
2. Add verification criteria to any prompt that lacked them.
3. Estimate: how many correction rounds would each rewrite have saved?

5. Thinking Together

Your prompts are precise, your CLAUDE.md is configured, and your tests pass. But every interaction follows the same pattern: you delegate, Claude executes, you verify. Something is missing—you are using Claude as a tool you configure, not a collaborator you think with.

The previous chapters answer "how do I tell Claude what to do?" This chapter answers a different question: how do I *think together* with Claude?

The core metaphor: Claude is a thinking mirror. It reflects your codebase back to you. Where the reflection is clear, your code is clear. Where it is distorted—where Claude misunderstands, guesses, or asks for clarification—you have found a documentation gap, a hidden assumption, or an unclear interface.

An Honest Caveat

Claude is a thinking partner with no memory, a tendency to agree, and occasional confident wrongness. The techniques in this chapter work *because* they structure the collaboration to counteract these weaknesses—not despite them.

Why This Chapter Exists

Every chapter so far answers "how do I configure Claude to do what I want?" This one answers "how do I use Claude to think more clearly?" The shift is from delegation to collaboration—from configure-delegate-verify to think-together-discover-build-better.

5.1. Hypothesis-Driven Debugging

When a bug appears, the default instinct is to paste the traceback and say "fix this." Claude will try—and often succeed on the first attempt. But for deeper bugs, this approach produces patches that address symptoms rather than root causes.

[Practitioner] Structure debugging as hypothesis testing, not error correction. Claude is a pair programmer who catches different errors because it has no sunk cost in your approach.

Debugging

Before:

```
Fix this error:
  File "feature_pipeline.py", line 47
  ValueError: negative value in non-negative
  feature 'days_since_last_purchase'
```

After:

```
I see a ValueError in feature_pipeline.py
line 47: negative values in a feature that
should be non-negative. Three hypotheses:

1. Log transform applied before clipping
   negative deltas
```

```
2. Currency conversion introduces negatives
   for returns/refunds
3. Timezone mismatch causes date subtraction
   overflow

For hypothesis 1: write a test with a purchase
  date after the reference date.
For hypothesis 2: write a test with a refund
  transaction in the events table.
For hypothesis 3: check if timestamps are
  timezone-aware or naive.

Run hypothesis 1 first---it is the most
likely given the stack trace.
```

The second prompt is longer but produces better outcomes: a root-cause diagnosis instead of a symptom patch. Each hypothesis becomes a testable prediction. When the test confirms one, you fix the cause, not the effect.

Tip
"Three hypotheses for this failure. Design a minimal test for each."

Key Insight
Rubber duck debugging works because articulating the problem reveals the gap. Claude-as-duck works better because the duck talks back—it asks questions you would not ask yourself, because you already "know" the answer. (Sometimes you are wrong about what you know.)

Tip
"Run this one test with this one input. Does it pass?" Isolation is the fastest path to root cause.

Try This
Pick a recent data pipeline bug—ideally one where results looked plausible but were wrong (e.g., feature drift, silent NaN propagation, leakage). Write three hypotheses for what caused it. Ask Claude to design a minimal test for each hypothesis. Would this approach have found the root cause faster?

5.2. Tests as Thinking Tools

In Chapter 7, tests serve verification: does the code do what it claims? Here, tests serve a different purpose—exploration: *what should the code do at all?*

Tests: Verification vs. Exploration
Before:
"Write tests for this function."
After:
"I am not sure what should happen at the boundary. Write 5 test cases exploring: empty input, single element, duplicates, negatives, overflow. Which behaviors surprise me?"

Why Tests Before Requirements
Writing test cases forces you to articulate expectations you have not stated. When a test case surprises you—the function does something you did not expect—you have discovered a requirement that was implicit. The test did not verify your code. It interrogated your assumptions.

Tip
"What invariant should always hold for this function, regardless of input?" This question surfaces design decisions hiding as implementation details.

5.2.1. Property-Based Tests as Assumption Detectors

[Practitioner] Property-based tests are particularly powerful as thinking tools. Instead of testing specific inputs, they test *invariants*—properties that must hold for all inputs. (These are the same property-based tests from the sixth layer of Chapter 7's validation architecture, applied here as a thinking tool rather than a verification gate.)

```
Prompt: "What properties should always be true
for this sort function?
- Output length == input length
- Output is ordered
- Output is a permutation of input
Write a property-based test for each."
```

When a property-based test fails on a randomly generated input, it has found an edge case neither you nor Claude anticipated. That failure is more valuable than a passing test—it reveals a boundary in your understanding.

5.3. Surfacing Hidden Assumptions

Every system rests on assumptions—about scale, about usage patterns, about what will never change. Most are invisible until they break. Claude can surface them, not because it is smarter, but because it has no investment in your preferred approach.

Cross-Ref
For test infrastructure and verification criteria, see Chapter 7. For characterization tests, see Chapter 11.

Architecture Review

Before:

"Review my database schema."

After:

"Here is my feature store schema. I designed it assuming: (1) features are computed daily in batch, not real-time, (2) training and serving use the same feature computation, (3) feature drift is monitored externally. Which assumption is most likely wrong in 6 months, and what breaks when it does?"

The second prompt names the assumptions explicitly, then asks Claude to stress-test them. This is not code review—it is assumption review.

The Pre-Mortem Prompt

"Imagine this feature has failed in production six months from now. What are the three most likely causes? Work backwards from failure to the design flaw that enabled it."

Pre-mortems are more effective than post-mortems because they cost nothing and can change the design. Ask Claude to run one before every architecture decision.

The Feynman Prompt

"Explain my auth flow as if I just joined the team and need to modify it. Where did you have to guess because the code does not make intent clear?"

Gaps in Claude's explanation are gaps in your documentation. What Claude

cannot explain, a new hire cannot understand.

5.4. Quick Wins That Compound

Tip
"What am I assuming that I have not written down?" The most dangerous assumptions are the ones nobody states.

These five techniques take minutes to apply and improve every subsequent Claude interaction. The metaphor: Claude reads your codebase like a developer who just joined the team. Good onboarding materials help Claude the same way they help humans.

5.4.1. Docstrings as Triple-Duty Documentation

A docstring serves three audiences simultaneously: you (in six months), Claude (right now), and your teammates (always).

Docstrings

Before:

```python
def calculate_risk(portfolio, window):
    scores = []
    for asset in portfolio:
        s = _score(asset, window)
        if s is not None:
            scores.append(s)
    return np.mean(scores) if scores else 0.0
```

After:

```python
def calculate_risk(
    portfolio: list[Asset],
    window: int = 30,
) -> float:
    """Average risk score across portfolio.

    Args:
        portfolio: Assets to evaluate.
            Empty portfolio returns 0.0.
        window: Lookback days. Default 30.

    Returns:
        Mean risk score in [0, 1].
        Returns 0.0 if no assets scorable.

    Note:
        Skips assets where _score() returns
        None (insufficient history).
        This is intentional, not an error.
    """
```

The "Note" section is the highest-value addition: it explains a design decision that would otherwise look like a bug. Claude will not misinterpret the `None` handling because the intent is documented.

5.4.2. Type Hints as Contracts

Five seconds to write, five minutes of debugging saved. `window: int = 30` tells Claude the type, the default, and the name—three constraints in five characters. Without the hint, Claude may pass a string, a float, or a

`timedelta`.

5.4.3. Error Messages That Debug Themselves

This connects directly to Principle 4 from Section 1.2: fail fast with diagnostics. An error message that includes what went wrong, what was expected, and what to do about it saves Claude the same investigation time it saves you.

5.4.4. Code Archaeology

For brownfield work, Claude makes an excellent excavation partner:

```
Prompt: "Why might the original author have
written this as a nested loop instead of a join?
What constraint explains this design choice?"
```

Instead of assuming the code is wrong, this prompt assumes the code is *explained by something you do not yet see*. Claude often finds the constraint—a database limitation, a legacy API, a performance requirement—that makes the original design rational.

5.4.5. README-Driven Development

Write the README first. Then ask Claude:

```
Prompt: "Read this README. What questions does a
new developer still have after reading it?"
```

Try This

Pick one function Claude frequently reads in your project. Add a docstring with Args, Returns, and one Note explaining a non-obvious design decision. Compare Claude's output on a task involving that function before and after the docstring.

Tip

"Read this function. What would you need to know before modifying it?" Gaps in the answer are gaps in your code.

5.5. Let Claude Interview You

[Official] For larger features, have Claude interview you before implementation. Start with a minimal prompt and ask Claude to dig into the hard parts.[1]

```
I want to build a feature drift monitoring system.
Interview me in detail. Ask about:
- Technical implementation
- Data sources and schemas
- Edge cases and failure modes
- Tradeoffs I might not have considered

Keep interviewing until we have covered everything,
then write a complete spec to SPEC.md.
```

[1] https://code.claude.com/docs/en/best-practices

Once the spec is complete, start a fresh session to implement it. The new session has clean context focused entirely on implementation, and you have a written spec to reference.

Why Interviews Beat Requirements Documents

You do not know what you do not know. When you write requirements, you only state what you have already thought of. When Claude interviews you, it asks about things you have not considered—edge cases, failure modes, infrastructure constraints. The result is more complete requirements with less effort.

5.6. Architecture Decision Records

When you make an architecture decision *with* Claude, the conversation captures not just what you decided but why, and what alternatives were considered. An Architecture Decision Record (ADR) preserves this reasoning for your future self.

```
# ADR-007: Offline Feature Computation

## Context
Feature computation runs in nightly batch.
Some features are stale by 12 hours at serving time.

## Decision
Keep batch for training features.
Add streaming for 3 real-time features
(last login recency, cart value, session count).

## Alternatives Considered
1. All streaming (rejected: roughly 10x infrastructure cost)
2. Faster batch, hourly (rejected: still stale)
3. Feature caching with TTL (rejected: cache
   invalidation complexity)

## Consequences
- Two feature computation paths to maintain
- Real-time features need monitoring for drift
- Training/serving skew possible for 3 features

## Assumptions to Revisit
- 3 real-time features sufficient for next 6 months
- Streaming infra handles peak load (Black Friday)
- Feature drift alerts catch training/serving skew
```

Key Insight

The most valuable ADR section is "Alternatives Considered." Claude suggests alternatives you would not—not because it is smarter, but because it has no investment in your preferred approach. A human colleague might hesitate to challenge your solution. Claude does not hesitate.

5.7. Quick Reference

- Claude is a thinking mirror: distortions reveal gaps in your code
- Debugging: three hypotheses, minimal tests, isolate blame

Cross-Ref

For plan documents and large-task workflows, see Section 6.7.

considered?" Enable extended thinking (Section 4.3) when drafting ADRs.

- Tests as exploration: discover requirements, not just verify them
- Interviews: let Claude ask what you have not considered
- Assumption surfacing: name your assumptions, then stress-test them
- Pre-mortem: imagine failure, work backwards to design flaw
- Docstrings, type hints, error messages: minutes to add, compound forever
- Code archaeology: assume intent before assuming error
- ADRs: capture alternatives considered, not just the decision

5.7.1. Building a Prompt Pattern Library

The techniques in this chapter can be codified as team-shared resources. The hypothesis debugging workflow becomes a custom command. The pre-mortem prompt becomes a line in your shared CLAUDE.md. The ADR template becomes a skill.

[Practitioner] Start personal, then share. Once a prompt pattern saves you time three sessions in a row, promote it from your terminal history to a command or skill that the whole team can use.

Try This

Choose one technique from this chapter and apply it today:

1. **Easiest**: Add a docstring to one feature engineering function Claude reads often. Include the expected DataFrame schema.
2. **Most impactful**: Use the hypothesis debugging workflow on your next pipeline bug.
3. **Most strategic**: Draft an ADR for an upcoming feature store or model serving decision.

Pick one. Do it now. The techniques compound—but only if you start.

Cross-Ref

For team-shared patterns and governance, see Chapter 14.

6. Context as Currency

Three hours into a session, Claude starts repeating itself. It forgets a rule you stated 20 minutes ago. It suggests an approach you already rejected. The code quality has noticeably dropped since the session started.

In Chapter 3, you learned two context commands: `/clear` and `/compact`. This chapter explains *why* context degrades, gives you a full toolkit for managing it, and introduces patterns for cross-session persistence.

6.1. Why Context Degrades

The Non-Linear Cost of Context

Context degradation is not gradual—there is a noticeable threshold where Claude begins to lose track of earlier instructions. Every token of context spent on irrelevant information (old errors, abandoned approaches, unrelated tasks) is a token unavailable for the task at hand.

The mechanism: as the context window fills, Claude's attention distributes across more tokens. Critical instructions (your CLAUDE.md rules, the current task description) compete for attention with accumulated noise. Past a threshold, signal-to-noise drops sharply.

Official

Performance degrades as context fills.
Use `/clear` between unrelated tasks.
Use `/compact` for selective summarization.
https://code.claude.com/docs/en/best-practices

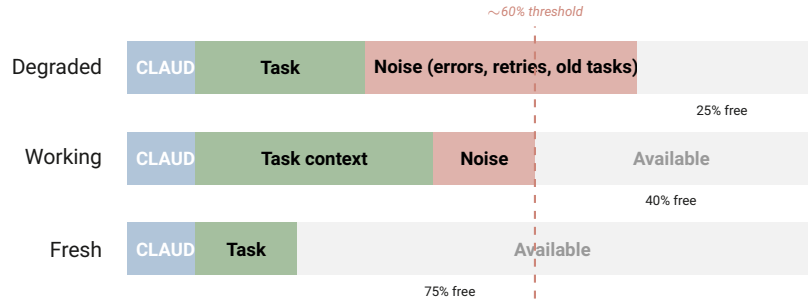

Figure 6.1.: Context composition over time (illustrative proportions). Noise (old errors, abandoned approaches) grows fastest, crowding signal. The ~60% threshold marks where quality drops non-linearly.

6.2. The Full Context Toolkit

`/clear`

Wipe context and start fresh. Use between unrelated tasks. Cost: ~30 seconds. Benefit: full attention on the next task.

`/compact`

Summarize conversation while preserving key decisions. Less aggressive than `/clear`; retains important context. Use when a long task genuinely needs accumulated context but the window is getting large.

`/rewind`

Checkpoint and roll back to a previous state. Every action Claude takes creates an automatic checkpoint. Double-tap `Esc` or run `/rewind` to open the rewind menu: restore conversation only, restore code only, restore both, or summarize from a selected message. Checkpoints

persist across sessions—you can close your terminal and still rewind later.

`/context`
: Visualize current context usage. Shows how much of the window is consumed and by what.

Subagents
: Delegate context-heavy research to isolated agents. Results return to your session; the research context does not. See Chapter 10 for the full treatment.

6.3. The Status Line: Zero-Cost Monitoring

[Official] Claude Code displays a configurable status line showing session metrics in real time.[1] By default it shows the active model, git branch, session cost, and context window usage percentage.

You can extend it with a custom script whose output is appended to the default display:

```
# ~/.claude/statusline.sh — runs locally, never sent to API
#!/bin/bash
echo "env: ${APP_ENV:-dev}"
```

The status line is your early-warning system for context degradation. When context usage enters the 60–70% range, proactively `/compact` or `/clear`—do not wait for quality to degrade before acting.

What to Watch

Context usage %
: The single most important metric. Watch for the 60–70% threshold where quality drops non-linearly.

Session cost
: Tracks spend in real time. Catches expensive operations before they compound.

Git branch
: Confirms you are working in the right context, especially when switching between worktrees (Section 10.3).

Model
: Confirms which model is active—important after `--model` overrides or skill-level model switches.

[Practitioner] The status line costs nothing and prevents the most common failure mode: context degradation discovered only after quality has already dropped. Enable it on day one.

Tip
Watch the token counter. When it approaches 60–70% of the window, proactively `/compact` or `/clear`. This threshold is a heuristic from observed

Official
The status line runs entirely locally—zero API tokens consumed. Fully customizable via script.
https://code.claude.com/docs/en/statusline

6.4. The Two-Failure Rule (Revisited)

This rule was introduced in Chapter 3. Now you understand *why* it works:

[1] https://code.claude.com/docs/en/statusline

[Practitioner] After two failed corrections, the context contains: the original error (~500 tokens), your first correction (~200 tokens), Claude's apology and retry (~800 tokens), your second correction (~200 tokens), and Claude's second attempt (~800 tokens). That is ~2,500 tokens of noise—nearly all of it encoding failure patterns rather than the actual task.

`/clear` discards 2,500 tokens of noise. A better initial prompt (informed by what went wrong) takes ~200 tokens and produces a better result.

6.4.1. Customizing Compaction

[Official] You can control what survives compaction by adding instructions to CLAUDE.md:[2]

```
## Compaction Rules
When compacting, always preserve:
- The full list of modified files
- Any test commands and their results
- Architecture decisions from this session
```

For partial compaction, use `Esc + Esc` or `/rewind`, select a message checkpoint, and choose **Summarize from here**. This condenses messages from that point forward while keeping earlier context intact—more surgical than `/compact`.

6.5. Session Management

[Official] Sessions persist across terminal restarts.[3] Key commands:

Official
Named sessions: `/rename`, `--continue`, `--resume`, `--from-pr`.
https://code.claude.com/docs/en/overview

`claude --continue`
: Resume the most recent session.

`claude --resume`
: Pick from a list of previous sessions.

`claude --from-pr 123`
: Start a session with PR context pre-loaded.

`/rename "feature-auth"`
: Give descriptive names for later retrieval.

6.5.1. Session Navigation

The session picker (invoked via `claude --resume`) supports keyboard navigation:

Key	Action
↑/↓	Navigate session list
`P`	Preview session content without opening
`R`	Rename session inline
`/`	Search sessions by name or content
`Enter`	Open selected session

[2] https://code.claude.com/docs/en/best-practices
[3] https://code.claude.com/docs/en/overview

Session Naming Conventions

Treat sessions like branches—one logical workstream per session. Use descriptive names that include the task type: `/rename "fix-null-handling-pipeline"`, `/rename "feat-recency-features"`, `/rename "debug-cv-leakage"`. The `--from-pr` flag is especially powerful for code review: `claude --from-pr 142` starts a session with the full PR diff and metadata pre-loaded, saving manual context setup.

6.5.2. The CURRENT_WORK.md Pattern

[Practitioner] Maintain a `CURRENT_WORK.md` file at project root. It provides 30-second context resume after any interruption.

Before & After

Before:

```
[No file. Developer spends 10 minutes re-discovering context.]
```

After:

```
## Right Now
Refactoring feature engineering to use
sklearn Pipelines.

## Why
Current feature code leaks test data during
cross-validation.

## Next Step
Write tests verifying no leakage across CV folds.

## Context When I Return
- Pipeline code in src/features/pipeline.py
- 3 of 7 transformers converted, 4 remaining
- Tests in tests/features/ — existing tests pass
```

6.6. Auto-Memory and Cross-Session Persistence

[Official] Claude maintains auto-memory that persists across conversations.[4] It records patterns, decisions, and lessons learned, making them available in future sessions without explicit re-prompting.

Tip

Update CURRENT_WORK.md at the start and end of every session. 60 seconds invested saves ~10 minutes of re-discovery.

Official

Claude records and recalls memories across sessions automatically. Memory files persist in `~/.claude/projects/*/memory/`.
https://code.claude.com/docs/en/memory

6.6.1. What to Store in Memory

Stable patterns
: Confirmed conventions that apply across sessions

Architecture decisions
: Why you chose X over Y (not just what)

Hard-won lessons
: Bugs, workarounds, gotchas that took time to discover

Known issues
: Pre-existing problems to avoid re-investigating

[4] https://code.claude.com/docs/en/memory

6.6.2. What NOT to Store

Transient state
: Current task details, in-progress work

Unverified conclusions
: Patterns seen once, not confirmed

Duplicates of CLAUDE.md
: Memory supplements configuration, does not replace it

Memory Entries

Before:

```
# Memory
The project uses pandas. We had a bug
once. Config is in settings.yaml. I like
using polars. Testing is important.
```

After:

```
# Architecture Decision
Feature pipelines use polars (not pandas)
since 2026-01. Migration complete for
src/features/. Tests still use pandas
fixtures — convert as encountered.

# Known Gotcha
scikit-learn ColumnTransformer silently
drops columns not in the transformer list.
Always verify output columns match schema
after pipeline.fit_transform().
```

Memory vs. CLAUDE.md vs. CURRENT_WORK.md

Three persistence mechanisms, each for a different purpose:

CLAUDE.md
: Project rules that apply every session. Actionable instructions.

Memory
: Lessons and patterns that emerged organically. Supplements CLAUDE.md.

CURRENT_WORK.md
: Transient state: what you are doing right now. Updated every session.

If a memory entry becomes a rule, promote it to CLAUDE.md and delete the memory. If a memory entry becomes stale (the bug was fixed, the pattern was abandoned), delete it—stale memories are noise.

[Practitioner] The test for whether something belongs in memory: "Will this still be true and useful in three months?" If yes, store it. If uncertain, wait for confirmation.

6.7. Plan Documents for Large Tasks

[Official] The recommended workflow for complex tasks: Explore → Plan → Implement → Commit.[5]

Official
Use Plan Mode (Shift+Tab) for read-only analysis before implementation.
https://code.claude.com/docs/en/overview

[5] https://code.claude.com/docs/en/best-practices

Plan Mode lets Claude analyze your codebase without modifying anything. Use it for architecture discussions, impact analysis, and strategy decisions.

[Practitioner] For tasks exceeding one hour or 500 lines of changes, create a plan document:

1. **Active** — currently being planned, open questions remain
2. **Ready** — plan approved, implementation can begin
3. **Implemented** — all phases complete
4. **Archived** — moved to `docs/plans/archived/`

Every plan document should include a "Decisions Made" section documenting: why specific implementations were chosen, alternatives considered but rejected, assumptions made, and tradeoffs accepted. This section is invaluable for postmortem analysis when a decision turns out to be wrong.

6.8. Quick Reference

Cross-Ref
For collaborative plan drafting and ADRs, see Chapter 5.

- Context degrades non-linearly—manage it actively
- `/clear` between tasks, `/compact` within long tasks
- `/context` to visualize usage, `/rewind` to undo missteps
- Two-failure rule: `/clear` + better prompt beats a third correction
- CURRENT_WORK.md for cross-session context at project root
- Auto-memory for cross-session patterns
- Plan Mode (Shift+Tab) before complex implementations

Try This

1. Create a `CURRENT_WORK.md` for your active project with Right Now, Why, Next Step, and Context sections. Include which pipeline step or notebook you are working on.
2. In your next session, check `/context` after 30 minutes. How much of the window is consumed?
3. Track correction rounds for one day. If any task exceeds two, note what a better initial prompt would have been.

7. The Edit-Test-Commit Loop

Claude just generated 200 lines of code that looks correct. The syntax is clean, the variable names are reasonable, the logic reads well. How do you know it actually works?

AI-generated code has a specific failure mode that human-written code does not: it *looks* correct. The appearance of correctness is precisely why verification is essential—the bugs are subtle, not obvious.

7.1. Verification Is King

[Convergence] "Give Claude a way to verify its work" is Anthropic's stated single highest-leverage practice.[1] This aligns exactly with what practitioners discover independently: the projects that succeed are the ones with verification at every layer.

Why Verification Is the Highest-Leverage Practice

Claude Code's creator, Boris Cherny, suggests that verification improves code quality by 2–3×—a plausible but hard-to-measure claim that aligns with practitioner experience. The mechanism is straightforward: when Claude can run tests, check types, and validate output against concrete criteria, it catches its own mistakes before you see them. The verification loop turns a single-pass generation into an iterative refinement process. Without verification criteria, you are the only quality gate. With them, Claude becomes a self-correcting system.

7.1.1. The 6-Layer Validation Architecture

Layer defenses so that no single failure mode goes undetected:

1. **Type Safety** — Catch type errors at compile/lint time. Python type hints with mypy, pandas-stubs for DataFrame types, pandera for schema validation.
2. **Input Validation** — Check preconditions at function entry. Fail fast with explicit errors (Chapter 1).
3. **Unit Tests** — Test each function in isolation. Happy path + error cases + edge cases. Target: 80%+ coverage for modules (phase-appropriate—see Section 7.2).
4. **Integration Tests** — Test multi-function workflows. Verify components work together with realistic data.
5. **End-to-End Tests** — Test complete user workflows from input to output. Verify the system meets requirements.
6. **Property-Based Tests** — Test invariants that should always hold. Generate random inputs. Catch edge cases you did not think of.

7.2. Phase-Appropriate Standards

Tip

Layers 1–2 are cheap to add to any project today. Layers 3–4 are the baseline for production. Layers 5–6 are for critical systems.

[1] https://code.claude.com/docs/en/best-practices

[Practitioner] Not all code needs the same rigor. Applying production standards to a prototype kills velocity. Applying prototype standards to production kills reliability.

Phase	Testing	Code Quality	Transition Criteria
Exploration	Manual OK	Long functions OK	Hypothesis validated
Development	Unit + integration	Style enforced, type hints	Coverage >60%, code review
Production	Full 6-layer	Strict lint, immutability	Coverage >80%, zero critical warnings

7.2.1. Making Phase Transitions Explicit

[Practitioner] Write phase transitions in CLAUDE.md:

```
## Phase: EXPLORATION (until March 15)
- Tests: manual OK for now
- Coverage target: none
- Graduation criteria:
  - [ ] Model validated on holdout set
  - [ ] No data leakage verified
  - [ ] Feature distributions documented
  - [ ] Dependencies locked
  When all checked: switch to DEVELOPMENT phase
```

The key insight: premature rigor kills exploration, but sloppy production kills credibility. The solution is not to choose one standard—it is to be explicit about which standard applies *right now* and when to transition.

7.3. Test-First with Claude

[Practitioner] The test-first pattern produces higher-quality code than code-first:

1. Describe the **interface**: inputs, outputs, edge cases.
2. Claude writes **tests** based on the interface.
3. Claude writes **implementation** that passes the tests.
4. Tests run **automatically** via hooks.

Official
"Give Claude a way to verify its own work." For greenfield, this means: provide test cases in your initial prompt, not after the code is written.
https://code.claude.com/docs/en/best-practices

```
Prompt: "Create a FeatureValidator class:
- Takes a DataFrame and a schema dict
- Validates column types, value ranges, null counts
- Returns ValidationResult with errors list
- Raises ValueError if required columns are missing

Write tests first, then implementation."
```

This works because Claude excels at test generation when given clear specifications. The tests then constrain the implementation, preventing the "looks correct but is subtly wrong" failure mode.

Cross-Ref
For tests as exploration tools—discovering requirements, not just verifying them—see Chapter 5.

7.4. Verification Criteria in CLAUDE.md

Before & After

Before:

```
## Testing
We use pytest. Tests are important.
```

After:

```
## Verification
- Always run tests after code changes
- Never commit without passing lint + tests
- Include edge case tests for every new function
- Coverage target: 80% for src/ modules
```

[Convergence] These lines in CLAUDE.md improve every interaction by giving Claude a concrete way to validate its own work.

7.5. Quality Gates via Hooks

Hooks enforce what CLAUDE.md can only suggest (see Chapter 8 for the full treatment). Three hooks that prevent quality debt:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/lint-on-write.sh",
        "description": "Auto-lint on write"
      }]
    }],
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/pre-commit-gate.sh",
        "description": "Block bad commits"
      }]
    }]
  }
}
```

Why Hooks Beat Instructions

CLAUDE.md: "Always run tests before committing." Claude may forget, especially in long sessions with degraded context.

Hook: `PreToolUse` matching `Bash` → gate on `git commit` commands, run `pytest` first. Tests run every time, regardless of context state. Cannot be forgotten.

The principle: if a standard is non-negotiable, make it a hook. If it is advisory, put it in CLAUDE.md.

7.6. Quick Reference

- AI-generated code *looks* correct—verification is essential
- 6-layer validation: types → input checks → unit → integration → E2E →

property

- Match rigor to phase: exploration → development → production
- Test-first: describe interface → tests → implementation
- Verification criteria in CLAUDE.md improve every session
- Hooks enforce non-negotiable standards

Try This

1. Add these three lines to your CLAUDE.md: "Always run `pytest tests/` after code changes. Never commit without passing `ruff check src/ && mypy src/`. Include edge case tests (empty DataFrames, NaN-heavy inputs) for every new function."
2. Add a PreToolUse hook (matching `Bash`) that gates commits on `pytest tests/ && mypy src/`.
3. Track: does Claude's test coverage improve over the next week?

8. Extending Claude: Commands, Skills, Hooks, and MCP

You keep running the same four commands manually: format, lint, test, commit. You explain the same deployment procedure every session. There must be a better way to encode these workflows so Claude executes them without being told each time.

Claude Code ships with four extension mechanisms—commands, skills, hooks, and MCP servers—forming a hierarchy of increasing autonomy. Understanding when to use each is the difference between a tool and a workflow.

8.1. Custom Slash Commands

[Official] Commands are Markdown files that expand into prompts when invoked.[1] They accept arguments via `$ARGUMENTS` and can reference files with `@path`.

Official
Place in `~/.claude/commands/` (global) or `.claude/commands/` (project). Invoked as `/command-name`.
https://code.claude.com/docs/en/best-practices

```
# .claude/commands/check.md
Run a comprehensive check of the project:
1. Run the test suite: `pytest tests/`
2. Run the linter: `ruff check src/`
3. Run the type checker: `mypy src/`
4. Report results as a summary table
```

[Practitioner] The best commands are not shortcuts—they encode decisions. Instead of a command that runs a test, create a command that decides *what to do next* based on project state.

```
# .claude/commands/next.md
Check the current project state:
1. Read CURRENT_WORK.md for context
2. Run `git status` and `git diff --stat`
3. Check for failing tests
4. Recommend the single highest-priority next action
```

8.2. Skills: Domain Knowledge and Workflows

Official
Custom slash commands have been merged into skills. Both `.claude/commands/` and `.claude/skills/` work. Skills take

Official
Each skill lives in its own directory as `SKILL.md`.
https://code.claude.com/docs/en/skills

[Official] Skills combine domain knowledge with executable workflows.[2] Unlike commands (which are prompt templates), skills can:

- Be applied automatically when Claude detects a relevant context
- Include file globs for automatic triggering
- Define multi-step procedures with validation gates
- Restrict available tools via `allowed-tools` frontmatter

[1] https://code.claude.com/docs/en/skills
[2] https://code.claude.com/docs/en/skills

- Run shell commands dynamically before sending content to Claude
- Spawn a forked subagent context via `context: fork`

8.2.1. File Structure and Precedence

Each skill is a directory containing a `SKILL.md` file:

```
.claude/skills/qa-health/SKILL.md      # Project skill
~/.claude/skills/deploy-check/SKILL.md # Personal (all projects)
```

[Practitioner] Precedence: enterprise (managed settings) > personal > project. Plugin skills use a namespaced format: `plugin-name:skill-name`.

8.2.2. Frontmatter Reference

All frontmatter keys are optional. The `name` defaults to the directory name if omitted.

Key	Purpose
`name`	Display name and `/slash-command`. Lowercase, hyphens, max 64 chars.
`description`	What the skill does. Claude uses this to decide when to auto-apply.
`allowed-tools`	Tools permitted without prompting while skill is active. **Hyphenated.**
`argument-hint`	Autocomplete hint, e.g. `[issue-number]`.
`model`	Override model while skill is active (`sonnet`, `opus`, `haiku`).
`context`	Set to `fork` to run in a forked subagent context.
`agent`	Which subagent type to use when `context: fork` is set.
`hooks`	Lifecycle hooks scoped to this skill's execution.
`user-invocable`	Set to `false` to hide from `/` menu. Default: `true`.
`disable-model-invocation`	Set to `true` to prevent Claude from auto-loading. Default: `false`.

8.2.3. Basic Skill Example

```yaml
---
name: qa-health
description: Run quality dashboard across all volumes
allowed-tools: [Bash, Read, Glob]
---
# QA Health Check
1. Run `python scripts/audits/health_dashboard.py`
2. Parse output for RED/YELLOW/GREEN status
3. Report findings with actionable recommendations
4. If any RED items, create TODO list for fixes
```

8.2.4. Dynamic Context

Official
The !`command` syntax runs shell commands at skill load time, injecting live data into the prompt.
https://code.claude.com/docs/en/skills

Skills can execute shell commands *before* their content reaches Claude. This injects live project state into the prompt:

```
---
name: review-changes
description: Review recent changes with full context
allowed-tools: [Bash, Read, Grep]
---
# Review Changes

Current branch status:
!`git status --short`

Recent commits:
!`git log --oneline -5`

Review the changes above for logic errors, missing tests,
and style violations per our CLAUDE.md.
```

Including "ultrathink" anywhere in skill content enables extended thinking for that skill's execution.

8.2.5. Forked Context: Skill as Subagent

[Practitioner] For skills that need isolation—long-running analysis, exploratory research, or tasks that should not pollute your main context—use `context: fork`.

```
---
name: deep-audit
description: Run thorough code audit in isolated context
context: fork
agent: general-purpose
model: opus
allowed-tools: [Read, Grep, Glob, Bash]
---
# Deep Audit
Analyze the entire codebase for:
1. Security vulnerabilities
2. Performance bottlenecks
3. Dead code
Report findings with file:line references and severity.
```

The forked context runs as a subagent—it gets its own context window, does not consume your main session's context, and returns a summary when done. This bridges skills and subagents (Chapter 10).

8.2.6. Budget and Permissions

Permission rules can target skills specifically:

```
{
  "permissions": {
    "allow": ["Skill(qa-health)"],
    "deny": ["Skill(deploy *)"]
  }
}
```

Official
Skill descriptions consume ~2% of context window (fallback: 16,000 characters). Use `/context` to check for excluded skills.
https://code.claude.com/docs/en/skills

`Skill(name)` matches exactly; `Skill(name *)` matches a prefix with any arguments; bare `Skill` denies all skill usage.

8.3. Hooks: Deterministic Automation

[Convergence] Hooks are the enforcement layer. Unlike CLAUDE.md instructions (which are advisory), hooks **always execute**.

Tip
A skill that saves 5 minutes per use and runs 3 times per week saves 13 hours per year. Measure value by frequency × savings.

Official
Configure via `/hooks` or in `settings.json`.
https://code.claude.com/docs/en/hooks

8.3.1. Available Lifecycle Events

Event	Matcher Target	Use Case
`SessionStart`	How session started	Load context, run health check
`UserPromptSubmit`	(no matcher)	Transform or validate user input
`PreToolUse`	Tool name	Validate inputs, block operations
`PermissionRequest`	Tool name	Auto-allow/deny by rule
`PostToolUse`	Tool name	Auto-lint, log results, trigger follow-up
`PostToolUseFailure`	Tool name	Error handling, retry logic
`Notification`	Notification type	Route to Slack, email, or dashboard
`SubagentStart`	Agent type	Initialize subagent-specific config
`SubagentStop`	Agent type	Process subagent results
`Stop`	(no matcher)	Run cleanup when Claude finishes
`TeammateIdle`	(no matcher)	Notify when agent team member idles
`TaskCompleted`	(no matcher)	Post-task automation
`ConfigChange`	Config source	React to settings file changes
`WorktreeCreate`	(no matcher)	Custom worktree setup logic
`WorktreeRemove`	(no matcher)	Custom worktree cleanup logic
`PreCompact`	Trigger type	Save state before context compaction
`SessionEnd`	Why session ended	Save state, update logs

[Practitioner] The golden rule: if a standard is non-negotiable, make it a hook. If it is advisory, put it in CLAUDE.md.

8.3.2. Notification Hooks

[Official] The `Notification` event fires when Claude needs human attention.[3] Use it for desktop alerts during long-running tasks:

```
{
```

[3] https://code.claude.com/docs/en/common-workflows

```
  "hooks": {
    "Notification": [{
      "hooks": [{
        "type": "command",
        "command": "notify-send 'Claude Code' 'Needs attention'",
        "description": "Desktop alert when Claude needs input"
      }]
    }]
  }
}
```

8.4. MCP: External Tool Integration

[Official] The Model Context Protocol (MCP) connects Claude to external services, databases, APIs, and custom tools.[4]

Tip
On macOS, replace `notify-send` with `osascript -e 'display notification ...'`. On Windows WSL, use `powershell.exe` with toast

Official
MCP provides a standardized way to connect Claude to external tools and data sources.
https://code.claude.com/docs/en/mcp

```
# Add an MCP server
claude mcp add github --transport stdio -- gh-mcp

# Scopes
# Local: current machine only (default)
# Project: .mcp.json committed to git (team-wide)
# User: ~/.claude/mcp.json (personal global)
```

Key concepts:

Servers
Provide tools, resources, and prompts via a standard protocol.

Tools
Functions Claude can call (query a database, create a ticket).

Resources
Data Claude can reference via `@server:resource/path`.

Dynamic loading
When tools exceed 10% of context (see Section 6.1 for why this matters), Claude uses on-demand search to keep context efficient.

[Official] Permission rules can use wildcards for MCP tool scoping:[5]

```
{
  "permissions": {
    "allow": ["mcp__github__*"],
    "deny": ["mcp__github__delete_*"]
  }
}
```

MCP Security
Verify server provenance before installation. Prefer servers from the official registry. Review permissions each server requires. Use Managed MCP for enterprise policy enforcement.

Tip
The `mcp__server__*` syntax allows all tools from a server. Combine with specific deny rules for fine-grained control.

[Practitioner] Before installing any MCP server, run this 3-step audit:

[4] https://code.claude.com/docs/en/mcp
[5] https://code.claude.com/docs/en/permissions

1. **Check the README and manifest.** What tools does the server expose? Does it request network access, filesystem access, or credentials? A GitHub MCP server should not need filesystem write access.
2. **Review requested permissions.** After `claude mcp add`, run `/permissions` to see what the server can do. Deny anything beyond its stated purpose. Use `mcp__server__*` allows with specific deny rules for destructive operations.
3. **Test in a sandboxed session.** Enable `/sandbox` or use a throwaway worktree for the first run. Verify the server behaves as documented before granting production access.

8.5. Plugins: Community Extensions

[Official] Plugins are pre-packaged extensions from the community and Anthropic.[6] They bundle multiple extension types into a single installable unit, avoiding manual configuration.

Cross-Ref
For Managed MCP and enterprise policy enforcement, see Section 14.2.

Official
Run `/plugin` to browse the marketplace. Plugins bundle skills, hooks, subagents, and MCP into installable units.
https://code.claude.com/docs/en/plugins

Code intelligence plugins

[Official] For typed languages, install a code intelligence plugin immediately.[7] These give Claude precise "go to definition" and "find references" capabilities, plus automatic error detection after edits. Supported languages include TypeScript, Rust, Go, Java, and Python (via Pyright). The improvement in code navigation accuracy is substantial.

Service integration plugins

Connect to external tools (Notion, Figma, monitoring) without writing MCP servers.

```
# Browse available plugins
/plugin

# Plugins appear in .claude/plugins/ after installation
```

[Practitioner] Plugins are the fastest path from "nothing configured" to "productive." Check the marketplace before building custom skills.

8.6. Sandboxing: Safe Autonomy

[Official] Sandboxing provides OS-level isolation that restricts filesystem and network access, allowing Claude to work more freely within defined boundaries.[8]

Official
Enable with `/sandbox`. Restricts filesystem and network access at the OS level.
https://code.claude.com/docs/en/sandboxing

With sandboxing enabled, Claude can run commands without per-action permission prompts because the sandbox prevents access beyond its boundaries. This is a safer alternative to `--dangerously-skip-permissions`.

8.7. CLI Tools: Context-Efficient Integration

[6] https://code.claude.com/docs/en/plugins
[7] https://code.claude.com/docs/en/common-workflows
[8] https://code.claude.com/docs/en/sandboxing

[Convergence] CLI tools are the most context-efficient way to interact with external services.[9]

Install the CLI for services you use regularly:

- GitHub: `gh` (issues, PRs, comments, releases)
- AWS: `aws` (S3, Lambda, deployment)
- GCP: `gcloud` (compute, storage, BigQuery)
- Monitoring: `sentry-cli`, `datadog-ci`

Claude knows how to use common CLIs. For unfamiliar ones, try: *"Use `tool --help` to learn about the tool, then use it to solve X."*

Tip
CLIs consume far less context than MCP tools or API calls. Prefer CLIs over MCP for well-known services.

8.8. The Delegation Hierarchy

Which Extension Mechanism?
Command — Simple prompt template, inline context. Use for: shortcuts, common prompts, decision-point commands.
Skill — Multi-step workflow with domain knowledge and tool restrictions. Use for: repeated procedures, quality dashboards, pre-release checks.
Hook — Deterministic automation on lifecycle events. Use for: non-negotiable standards, auto-formatting, test gates.
MCP — External service integration. Use for: databases, APIs, third-party tools, cross-project knowledge bases.
Rule of thumb: use the simplest mechanism that solves the problem.

8.9. Visual: The Delegation Hierarchy

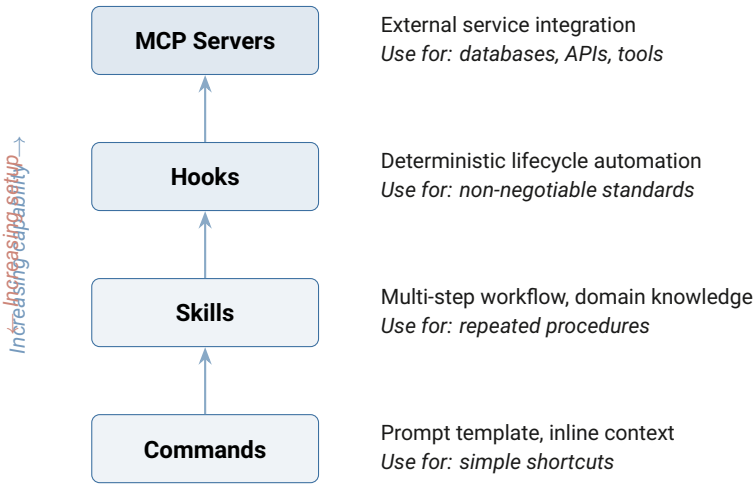

Figure 8.1.: Match extension mechanism to task requirements. The simplest mechanism that solves the problem is the best choice.

Key Insight
M

ost developers jump to complex solutions when a simple one would suffice. A command handles 80% of workflow needs. Add skills, hooks, and MCP only when the simpler mechanism is genuinely insufficient. Chapter 10

[9] https://code.claude.com/docs/en/best-practices

explores subagents and parallel delegation patterns for when even MCP is not enough.

Try This

Create a `/check` command for your project:

1. Create `.claude/commands/check.md`.
2. Include: `pytest tests/`, `ruff check src/`, `mypy src/`.
3. Format output as a summary table: Check | Status | Details.
4. Test it: type `/check` in your next session.

Bonus: add a PostToolUse hook matching `Edit|Write`: auto-lint written files.

Part III.

Advanced Craft

You are at Level 3 (Systematic)—you manage context, test rigorously, and use commands and hooks. Advanced patterns take you toward Level 4 (Autonomous): architecting configurations that scale across projects, delegating work to agents, and managing entire project lifecycles with Claude.

9. CLAUDE.md Architecture

You have five projects. Each has its own CLAUDE.md with 80% identical content. One project's CLAUDE.md has grown to 600 lines and Claude ignores half the rules. You need architecture, not just configuration.

In Chapter 2, you created your first CLAUDE.md. Now that you have experience with multiple projects, we will explore the architecture behind configurations that scale.

9.1. The Hub-and-Spoke Pattern

[Practitioner] For developers managing multiple repositories, maintain a central "hub" with shared patterns referenced from each project.

Before & After

Before:

Five CLAUDE.md files, each 200 lines, 80% identical. Updating git conventions means editing five files. Three of them are out of date.

After:

One hub with shared patterns. Five spokes, each 40 lines. Update once, all projects inherit.

```
# Project CLAUDE.md (spoke — 40 lines)
@~/shared-patterns/git.md
@~/shared-patterns/testing.md

## Project-Specific
- Build: `make digital`
- Test: `pytest tests/`
- Architecture: Tufte-style LaTeX, one chapter per file
```

[Official] Use the `@path/to/file.md` import syntax to reference external files without duplicating content.[1]

Why Hub-and-Spoke

Duplication across CLAUDE.md files is not just inconvenient—it is actively harmful. When the git convention changes in one file but not others, Claude learns different conventions depending on which project it is in. Inconsistency accumulates until you cannot trust that any project's instructions are current.

9.2. Tier Classification

Not every project needs the same level of configuration. Classify by complexity:

[1] https://code.claude.com/docs/en/memory

Tier	Components	When to Use
T0	None	Prototypes, learning repos
T1	CLAUDE.md only	Small scripts, documentation projects
T2	CLAUDE.md + settings	Standard development projects
T3	+ skills, commands, rules	Multi-module projects with workflows
T4	+ hooks, MCP	Projects requiring automation and quality gates
T5	+ agent teams, pipelines	Enterprise-scale projects with CI/CD

9.3. Conditional Rules

Tip
Start at T1. Promote when friction justifies it. Most solo projects stabilize at T2–T3.

[Official] Place `.md` files in `.claude/rules/` with `paths:` frontmatter to control when they load.[2]

Rules without `paths:` frontmatter load on *every* session. For monorepos and multi-module projects, this bloats context with irrelevant instructions.

```
---
paths: notebooks/**/*.py
---
# Notebook Rules
- No hardcoded file paths (use config)
- Every notebook has a markdown header explaining purpose
- Run `nbstripout` before committing
```

```
---
paths: src/models/**/*.py
---
# Model Training Rules
- Log all hyperparameters to MLflow
- Save model artifacts to models/
- Include reproducibility seed in all configs
- Run `pytest tests/models/` after changes
```

Common Mistake
Forgetting `paths:` frontmatter. The rule loads globally, Claude's context fills with irrelevant instructions, and important rules from CLAUDE.md get deprioritized.

9.4. Settings: 5-Level Precedence

Claude Code uses a 5-level settings hierarchy. Higher levels override lower:

Official
Settings documentation:
https://code.claude.com/docs/en/settings

1. **Managed** (`managed-settings.json`) — IT-deployed, highest priority
2. **CLI arguments** — temporary session overrides
3. **Local** (`.claude/settings.local.json`) — personal, auto-gitignored
4. **Project** (`.claude/settings.json`) — team standards, committed to git
5. **User** (`~/.claude/settings.json`) — global personal defaults

[2] https://code.claude.com/docs/en/best-practices

[Practitioner] Convention: commit `settings.json` for team-wide standards. Keep `settings.local.json` for personal preferences (model choice, output style).

9.5. CLAUDE.md Length and Attention

[Practitioner] When CLAUDE.md grows long:

1. Move module-specific rules to `.claude/rules/` with `paths:`
2. Move shared patterns to a hub via `@path` imports
3. Move enforcement rules to hooks (they cannot be ignored)
4. Keep core CLAUDE.md to project-wide essentials: build commands, architecture, verification criteria

Warning
Keep core CLAUDE.md under ~300 lines. Beyond that, attention per rule tends to decrease. This is heuristic, not a hard threshold—but the effect is real. (Total config including `@imports` may reach 500 lines safely since imports load only when relevant.)

9.6. Output Styles

[Official] Output styles customize how Claude structures its responses: tone, verbosity, formatting conventions, and problem-solving protocols.[3]

[Practitioner] Custom styles are most valuable when they encode *decision protocols*, not just formatting preferences. A style that says "use bullets" saves minimal effort. A style that says "classify the issue type before searching for solutions" changes the quality of every interaction.

Official
Configure via `outputStyle` in settings or `--output-style` flag.
https://code.claude.com/docs/en/best-practices

9.7. Visual: The Configuration Hierarchy

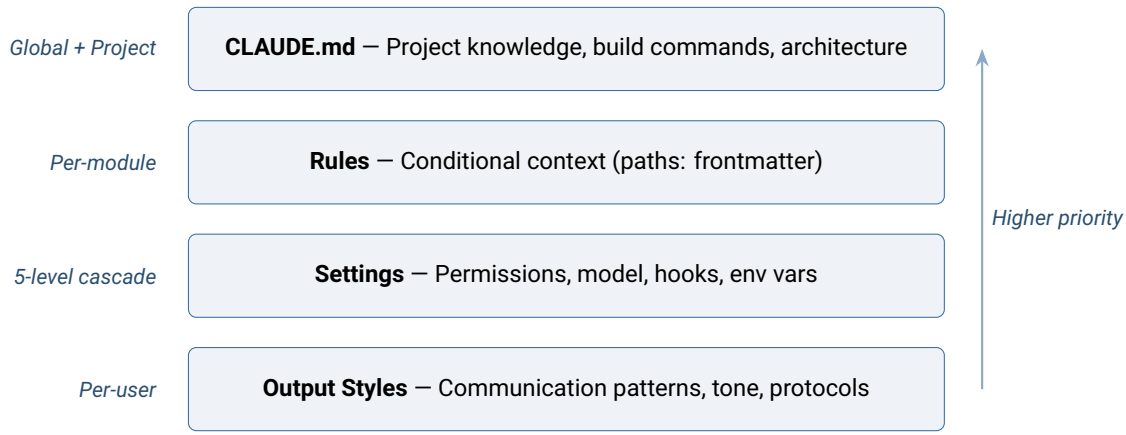

Figure 9.1.: The four configuration layers, each handling a distinct concern.

Key Insight
T

he configuration hierarchy separates concerns: CLAUDE.md for project knowledge, rules for conditional context, settings for permissions and behavior, output styles for communication. Mixing these concerns (e.g., putting permission rules in CLAUDE.md) reduces their effectiveness.

[3] https://code.claude.com/docs/en/best-practices

Try This

If you have 3+ projects:

1. Create a shared patterns directory (`~/shared-patterns/`).
2. Move your git commit conventions and testing standards there.
3. Replace the duplicated content in each project's CLAUDE.md with `@~/shared-patterns/git.md` imports.
4. Verify: each project's CLAUDE.md should be under 100 lines.

If you have one project: count the lines in your CLAUDE.md. If over 300, identify 3 sections that should be conditional rules.

10. Agents, Delegation, and Parallel Work

A code review needs four perspectives: correctness, security, performance, and style. Running them sequentially takes 20 minutes. Running them in parallel takes 5. But when should you delegate, and when should you just do it yourself?

In Chapter 8, you met subagents briefly as part of the extension hierarchy. This chapter covers the full delegation system: when to use it, how to design agent workflows, and how to run work in parallel.

10.1. Subagents: Isolated Context Workers

[Official] Subagents are purpose-built Claude instances that run in isolated context.[1] They receive a task, execute it with their own tool set, and return results to the parent session.

Vocab
In this handbook, **subagent** means an isolated worker with its own context. **Agent team** means a collaborative group that shares a task list and can message each other.

```
# .claude/agents/data-quality-reviewer.md
---
name: data-quality-reviewer
description: Review data pipeline for quality issues
tools: [Read, Grep, Glob]
---
You are a data quality specialist.
Review the provided pipeline code for:
1. Schema violations and type mismatches
2. Null handling patterns (silent drops
   vs explicit imputation)
3. Distribution drift between train and serve
4. Data leakage between train and test splits
Report findings with severity ratings
(Critical/High/Medium/Low).
```

10.1.1. Built-In Subagents

Claude Code ships with several built-in subagent types that can be invoked via the Task tool without custom definitions:

Tip
Note the frontmatter key is `tools` (not `allowed_tools`). Skills use `allowed-tools` (hyphenated). The inconsistency is intentional—they follow different specification standards.

Explore
: Fast codebase exploration—find files, search code, answer questions.

Plan
: Read-only analysis for implementation planning.

Bash
: Command execution specialist.

General-purpose
: Multi-tool agent for complex tasks.

[1] https://code.claude.com/docs/en/sub-agents

10.2. When to Delegate

Delegate or Do Directly?

Delegate when:

- The task is independent of your current work
- It requires heavy context that would pollute your main session
- Multiple tasks can run in parallel for wall-clock savings
- You need an unbiased second opinion (writer/reviewer pattern)

Do directly when:

- The task takes under 2 minutes
- It depends on your current session's context
- Sequential reasoning is required (step B needs step A's result)
- The task is too simple to justify the overhead

10.3. Git Worktrees: Isolated File Systems

Worktrees solve the fundamental problem of parallel file modifications: two agents editing the same files will conflict. Worktrees give each agent its own isolated copy of the repository.

Cost

Each subagent consumes its own context tokens. 4 parallel agents = 4× tokens, but only 1× wall-clock time. Parallelism trades cost for speed.

Official

The `--worktree` flag creates an isolated copy of your repository for

Before & After

Before:

```
# Manual: create worktree, remember to clean up
git worktree add ../feature-cache -b feature-cache
cd ../feature-cache && claude
# ... work ...
cd - && git worktree remove ../feature-cache
```

After:

```
# Automatic: Claude manages the worktree lifecycle
claude --worktree "Implement caching layer"
claude --worktree "Add rate limiting"
```

Each worktree gets its own git branch and a complete isolated copy of the repository. Cleanup is automatic: if no changes are made, the worktree is removed silently. If changes exist, you are prompted to merge or discard.

10.3.1. Subagent Worktrees

[Official] Subagents can also run in isolated worktrees using the `isolation` frontmatter key:[2]

```
# .claude/agents/parallel-reviewer.md
---
name: parallel-reviewer
tools: [Read, Grep, Glob]
isolation: worktree
---
Review the code for correctness and edge cases.
```

[2] https://code.claude.com/docs/en/sub-agents

```
Report findings with file:line references.
```

When `isolation: worktree` is set, the subagent operates on its own copy of the repository. This is essential for agents that modify files—without isolation, two agents editing the same file will produce conflicts.

Worktree Housekeeping
Add `.claude/worktrees/` to your `.gitignore`. Claude stores worktree metadata there; it should not be committed. If orphaned worktrees accumulate (e.g., after a crash), run `git worktree prune` to clean them up.

Cost
Worktrees trade disk space for isolation. Each worktree shares git objects but duplicates the working tree. A 500 MB repo spawning 4 worktrees may use ~1 GB additional disk. Run `git worktree prune` to clean up stale entries.

10.4. The Writer/Reviewer Pattern

[Official] Use separate sessions to avoid confirmation bias:[3]

1. **Writer session**: Makes changes based on requirements.
2. **Reviewer session**: Reviews the diff without knowledge of intent. "Review this diff for bugs, regressions, and missed edge cases. You have no context about why these changes were made."

This pattern catches errors that the writer is blind to because of its accumulated context about the "correct" approach.

[Practitioner] Extend to multi-perspective review with subagents:

1. **Correctness agent**: "Does this logic do what it claims?"
2. **Data quality agent**: "Schema violations? Null patterns? Drift?"
3. **Statistical validity agent**: "Are assumptions met? Distributions stable?"
4. **Reproducibility agent**: "Seeds set? Configs versioned? Results deterministic?"

Running four agents in parallel takes the same wall-clock time as one.

Cross-Ref
Think together first (Chapter 5), then separate for independent review.

10.5. Agent Teams (Research Preview)

[Official] Agent Teams extend subagents with collaboration: shared task lists, direct messaging between agents, and team lead orchestration.[4]

Best suited for:

- Research tasks requiring multiple perspectives
- Cross-functional coordination (feature engineering + model training + evaluation + data validation)
- Complex multi-module changes that need coordination

[Practitioner] Prefer parallel subagents for independent tasks that share no state. Use Agent Teams when agents need to coordinate mid-task—e.g., one agent discovers a schema change that another agent must respect.

Official
Research preview. Multiple agents share a task list and can message each other.
https://code.claude.com/docs/en/agent-teams

Research Preview
Agent Teams are a research preview feature. The API and behavior may change. Use for exploration, not production workflows.

[3] https://code.claude.com/docs/en/best-practices
[4] https://code.claude.com/docs/en/agent-teams

10.6. Quick Reference

- Subagents: isolated context, independent tools, return results to parent
- Subagent frontmatter key: `tools` (not `allowed_tools`)
- Worktrees: isolated file systems for parallel file modifications
- Writer/Reviewer: separate sessions prevent confirmation bias
- Delegate when: independent, context-heavy, parallelizable, needs fresh perspective
- Do directly when: quick, context-dependent, sequential, simple
- Agent Teams: research preview for multi-agent collaboration

Key Insight

P

arallelism in Claude Code is not about speed alone—it is about *perspective isolation*. A reviewer who cannot see the writer's reasoning catches different bugs. A security agent that only sees code (not intent) finds different vulnerabilities. The diversity of perspectives is the value, not just the time savings.

Try This

Identify one review task that currently takes you more than 10 minutes. Design a 2-agent parallel split:

1. Agent 1: data quality review (schema, nulls, distributions).
2. Agent 2: statistical validity review (leakage, assumptions, reproducibility).
3. Are the tasks truly independent? (If Agent 2 needs Agent 1's output, they cannot run in parallel.)
4. Write the agent definition file for one of them.

11. Starting and Refactoring Projects

Two situations: a blank directory for a new project, and a 50,000-line legacy codebase with zero tests. Both need Claude's help. Both need different protocols.

This chapter unifies two project lifecycle phases: greenfield (starting from scratch) and brownfield (introducing Claude to existing code). Both follow the same principle: incremental investment, with each step independently valuable. The testing patterns from Chapter 7 and the extension mechanisms from Chapter 8 both play central roles.

11.1. Greenfield: From Zero to Production

11.1.1. Day 1: Foundation

[Convergence] Three investments on day one compound into dramatically better outcomes over the project's lifetime:

1. **Run `/init`** to generate a starter CLAUDE.md. Claude examines your project structure and proposes conventions.
2. **Set up permissions** in `.claude/settings.json`. Deny access to `.env*` and `secrets/`.
3. **Add one hook**: `PostToolUse` matching `Edit|Write` → auto-format. This prevents style drift from the first file onward.

11.1.2. Week 1: Patterns Emerge

1. **Add testing infrastructure.** Configure test runner in CLAUDE.md.
2. **Add your first custom command** for the most common workflow.
3. **Configure PreToolUse hook** matching `Bash` → gate `git commit` on passing tests. The single most valuable hook for new projects.

11.1.3. Month 1: Stabilization

1. **Review and trim CLAUDE.md.** Remove instructions Claude consistently follows without being told. Add instructions for rules Claude keeps violating.
2. **Transition from exploration to development phase** (Section 7.2). Add explicit graduation criteria to CLAUDE.md.
3. **Add skills** for repeated multi-step workflows.
4. **Set up memory** for cross-session persistence.
5. **Document architecture** as actionable instructions, not prose.

Tip
Most solo projects stabilize at Tier 2–3 within one month. Resist the urge to jump to Tier 5.

11.2. Brownfield: Introducing Claude to Existing Code

11.2.1. Start with Understanding, Not Action

[Official] Before any changes, use Plan Mode (Shift+Tab) to let Claude explore.[1] Ask:

- "Explain the architecture of this project."
- "What patterns does this codebase use?"
- "Where are the tests? What is the coverage?"

[Official] Document existing conventions in CLAUDE.md, not aspirational ones.[2] A CLAUDE.md that says "all features have unit tests" when most feature code is untested notebook-extracted scripts will cause inconsistent behavior—some features tested, some not.

Official
Use Plan Mode first—let Claude read and understand the codebase without modifying anything.
https://code.claude.com/docs/en/overview

Cross-Ref
For code archaeology techniques in brownfield projects, see Chapter 5.

11.2.2. Incremental Adoption

[Practitioner] Adopt Claude configuration in four steps, each independently valuable:

1. **CLAUDE.md** documenting current state (conventions as they are)
2. **Permissions** restricting Claude to safe operations
3. **Hooks** enforcing existing standards (not new ones)
4. **Skills** emerging organically as patterns stabilize

11.3. The Incremental Refactoring Protocol

[Practitioner] Never attempt a big-bang rewrite (this is anti-pattern #7 from Chapter 12). Use four-step increments, each independently shippable:

1. Extract
Isolate the function or module to refactor. Create a clear boundary between "changing" and "not changing."

2. Test
Write characterization tests capturing current behavior. Claude excels here—prompt: "Write tests that document exactly what this function does today, including edge cases."

3. Harden
Refactor with tests as safety net. Each change verified against the characterization tests.

4. Promote
Move refactored code into production. Run full regression suite. Verify no downstream breakage.

Session Sizing
Each step fits in a 25-minute focused session. Commit after each step. If interrupted, the project is always in a working state.

11.3.1. Characterization Tests

[Convergence] Before changing any code, have Claude write tests that

[1] https://code.claude.com/docs/en/overview
[2] https://code.claude.com/docs/en/best-practices

capture existing behavior.

```
Prompt: "Read src/features/legacy_features.py.
Write pytest tests that document its exact
current behavior:
- What DataFrames does it accept?
- What columns does it produce?
- What happens with NaN inputs?
Do NOT write tests for how it SHOULD work.
Write tests for how it DOES work."
```

11.3.2. Golden Master Testing for Pipelines

[Practitioner] For data pipelines, a golden master captures the exact output of the original code so that refactored code can be verified against it.

1. **Capture**: Run the original pipeline on sample data. Save the output DataFrame as a fixture (CSV or Parquet).
2. **Compare**: New code must produce identical output within floating-point tolerance.
3. **Verify**: Use `pd.testing.assert_frame_equal(new, golden, atol=1e-6)`.

Tip
Characterization tests are temporary—they may test buggy behavior. That is intentional. They exist to detect *changes*, not to assert *correctness*.

```
Prompt: "Run legacy_build_features() on
tests/fixtures/sample_orders.csv.
Save the output DataFrame to
tests/fixtures/golden_features.parquet.
Then write a test that runs the new
build_features() on the same input and asserts
pd.testing.assert_frame_equal(
    new, golden, atol=1e-6)."
```

When to use: Refactoring feature pipelines, migrating notebook code to modules, replacing pandas with polars.

Golden Masters Test Behavior—Not Correctness
A golden master preserves the original output—including any bugs. If the old code computed a feature incorrectly, the golden master encodes that incorrect behavior. Fix bugs *after* migration, with explicit tests for the fix. The golden master's job is to guarantee that the migration itself changed nothing.

Characterization Tests vs. Golden Masters
Characterization tests: for individual functions. "Document exactly what this function does today." Best when: refactoring a function's internals without changing its API.
Golden masters: for pipelines and multi-step workflows. "Capture the exact output of this pipeline on sample data." Best when: migrating pipeline code (e.g., pandas to polars, notebook to module).
Both: during pipeline migration. Characterization tests verify each function; golden masters verify the composed pipeline produces identical output.

11.4. Visual: The Incremental Refactoring Cycle

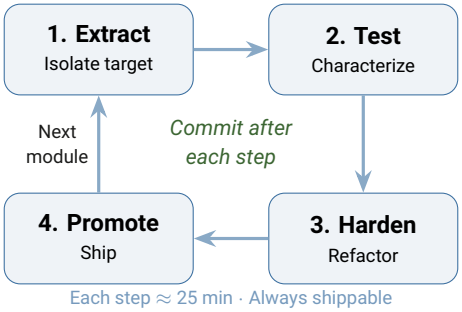

Figure 11.1.: The incremental refactoring cycle. Each step is independently shippable.

Tip
Use `atol/rtol` for floats, exact match for categoricals, `check_like=True` for column-order independence.

11.5. The Quality Flywheel

[Practitioner] After each refactoring sprint:

1. Update CLAUDE.md with lessons learned
2. Add new hooks for discovered gotchas
3. Refine skills based on what worked
4. Update metrics baselines

Each sprint improves not just the code but the tooling around the code. After three sprints, the refactoring process itself is significantly faster. For large codebases, consider parallel refactoring across independent modules using git worktrees (Section 10.3).

Key Insight
R

efactoring with Claude follows the same principle as refactoring without it: small, verifiable increments. The difference is that Claude can write characterization tests in minutes instead of hours, making the Extract → Test → Harden → Promote cycle dramatically faster.

Try This
Pick the messiest feature engineering function in your project (under 100 lines).

1. Have Claude write characterization tests without telling it what the function *should* do. Just: "Document what DataFrames it accepts, what columns it produces, and what happens with NaN inputs."
2. Run the tests. Do they pass?
3. If yes: save a golden master fixture and you have a safety net for refactoring. If no: you have discovered undocumented behavior. Either way, you have learned something valuable.

12. Anti-Patterns and Recovery

Three hours in, Claude is hallucinating function names and ignoring your CLAUDE.md rules. What went wrong? More importantly: how do you escape?

Every tool has characteristic misuse patterns. These eight anti-patterns are the most common ways developers undermine their productivity with Claude Code. Each includes the symptom, root cause, prevention, *and a recovery guide for when you are already stuck*.

12.1. Context Overload

Anti-Pattern: Context Overload

Symptom: Claude ignores important rules. Instructions from CLAUDE.md are followed inconsistently. Behavior degrades over long sessions.

Prevention: Hub-and-spoke architecture (Chapter 9). Conditional rules with `paths:` frontmatter. Hooks for non-negotiable standards. Keep core CLAUDE.md under ~300 lines.

Root cause: CLAUDE.md has grown to contain every rule, convention, and preference accumulated over months. Claude's attention dilutes across all of it, and critical rules get the same weight as minor preferences.

Before & After

Before:

```
# CLAUDE.md — 650 lines
Includes: git conventions, code style, architecture notes,
deployment procedures, 12 module-specific rules, personal
preferences, historical decisions, aspirational goals...
```

After:

```
# CLAUDE.md — 80 lines (core project rules)
# .claude/rules/backend.md — paths: backend/**/*.py
# .claude/rules/frontend.md — paths: frontend/**/*.tsx
# @~/shared-patterns/git.md — imported
# Hooks enforce: formatting, tests, lint
```

Recovery: Context Overload

You are here: Claude is ignoring rules that are in your CLAUDE.md. Instructions work at session start but fail after 30 minutes.

Escape route:

1. Count lines in CLAUDE.md. If over 300: split.
2. Move module-specific rules to `.claude/rules/` with `paths:`.
3. Move shared patterns to hub imports.
4. Move non-negotiable standards to hooks.
5. What remains should be project-wide essentials only.

12.2. The Kitchen Sink Session

Anti-Pattern: Kitchen Sink Session

Symptom: Performance degrades midway through a session. Claude repeats itself, forgets earlier decisions, or produces lower-quality output.

Prevention: Use `/clear` between unrelated tasks. One session per logical task. Use `/compact` if the task needs accumulated context.

Root cause: Multiple unrelated tasks in a single session. Each task's context remains, consuming tokens that contribute nothing to the current task.

Recovery: Kitchen Sink Session

You are here: You are 90 minutes into a session that started with debugging, moved to feature work, and now involves documentation. Claude's responses are noticeably worse than at session start.

Escape route: Stop. `/clear`. Start a fresh session for the current task only. Before clearing, note your current state in CURRENT_WORK.md so you do not lose context about what you were doing.

12.3. Over-Correcting

Anti-Pattern: Over-Correcting

Symptom: Three or more rounds of "no, that's not what I meant" followed by increasingly desperate attempts at the same task.

Prevention: Two-failure rule (Section 6.4): after two failed corrections, `/clear` and write a better initial prompt.

Root cause: Each correction adds noise—the original error, your correction, Claude's acknowledgment, and its retry. After three rounds, the context is dominated by failure patterns.

Recovery: Over-Correcting

You are here: You are on correction round 4. Claude keeps making the same mistake with slight variations.

Escape route:

1. `/clear` immediately.
2. Before re-prompting, write down exactly what you want (on paper or in a scratch file).
3. Include: what the function should do, an example input/output pair, and one specific constraint Claude kept violating.
4. Re-prompt with this more precise request.

Cross-Ref

The hypothesis debugging workflow in Chapter 5 prevents over-correction by structuring the problem before the first prompt.

12.4. The Permanent Prototype

Anti-Pattern: Permanent Prototype

Symptom: Code has been "working" for weeks but has no tests, no type hints, no error handling. "We'll add those later" becomes permanent.

Prevention: Define explicit phase transition checklists in CLAUDE.md (Section 7.2). Set a date for graduation from exploration to development.

Root cause: The exploration phase has no defined exit criteria. Without

an explicit transition, the code accumulates users and dependencies while remaining at prototype quality.

Recovery: Permanent Prototype

You are here: Your project has been in "exploration" mode for 6 weeks. It has users. It has no tests.

Escape route:

1. Add a DEVELOPMENT phase to CLAUDE.md today, effective immediately.
2. Have Claude write characterization tests (Section 11.3) for the 3 most critical functions.
3. Add a PreToolUse hook (matching `Bash`) that gates `git commit` on those tests passing.
4. You now have a safety net. Refactor incrementally from here.

12.5. Pipeline Direction Violations

Anti-Pattern: Pipeline Direction Violations

Symptom: Carefully edited downstream artifacts are overwritten when an upstream script is re-run.

Prevention: Document pipeline flow direction. Mark generated files: `# GENERATED --- do not edit manually`.

Root cause: Editing a generated file, then re-running the generator. The generator regenerates from source, destroying manual edits.

Recovery: Pipeline Direction Violations

You are here: You just ran `extract_features.py` and it overwrote 2 hours of manual YAML edits.

Escape route:

1. `git checkout -- path/to/overwritten/files` to restore.
2. Add `# GENERATED --- do not edit manually` to generated files.
3. If manual edits are needed, edit the *source* (.tex, .py) and re-generate. Never edit downstream.

12.6. The Verification Gap

Anti-Pattern: Verification Gap

Symptom: AI-generated code accepted without testing. Subtle bugs surface in production weeks later.

Prevention: Always provide verification criteria (Section 7.1). Tests with code—never after and separately. Hook-enforced test requirements.

Root cause: AI-generated code *looks* correct. The syntax is clean, variable names are reasonable, the logic reads well. This appearance of correctness is precisely why verification is essential.

Recovery: Verification Gap

You are here: You have 1,500 lines of untested AI-generated code. It "works" but you have zero confidence in it.

Escape route:

1. Do NOT try to test everything at once.

2. Identify the 3 most critical functions (highest risk, most dependencies).
3. Have Claude write tests for those 3 functions only.
4. Add a PreToolUse hook gating commits on tests. No new untested code.
5. Expand coverage incrementally: 3 more functions per week.

12.7. The Infinite Exploration

Anti-Pattern: Infinite Exploration

Symptom: You ask Claude to "investigate" something without scoping it. Claude reads hundreds of files, filling the context window with exploration results that crowd out implementation.

Prevention: Scope investigations narrowly. Use subagents for exploration so the research context does not consume your main session (Chapter 10).

Root cause: Unbounded investigation prompts like "look into how auth works" give Claude no stopping criteria. It reads every related file, consuming context that should be reserved for the actual task.

Before & After

Before:

```
Investigate how our authentication system works.
```

After:

```
Use a subagent to investigate how our auth system
handles token refresh, and whether we have any
existing OAuth utilities I should reuse.
Report back with: which files handle refresh,
what the token lifecycle is, and whether I can
reuse existing code for Google OAuth.
```

Recovery: Infinite Exploration

You are here: You asked Claude to "look into" something and it has read 40 files. Your context is full of exploration results and you have not started the actual task.

Escape route:

1. `/clear` immediately.
2. Re-prompt with a scoped question and explicit deliverables.
3. Or: delegate to a subagent so exploration runs in isolated context.

12.8. Big-Bang Refactoring

Anti-Pattern: Big-Bang Refactoring

Symptom: A large rewrite fails partway through, leaving the codebase broken.

Prevention: Incremental protocol (Section 11.3): Extract → Test → Harden → Promote. Each step independently shippable. Commit after each step.

Root cause: The scope of the rewrite exceeds what can be held in context. Claude loses track of the original behavior and introduces regressions.

Recovery: Big-Bang Refactoring

You are here: You are 3 hours into a rewrite. Half the tests fail. The old code and new code are mixed together.

Escape route:

1. `git stash` the current mess.
2. Return to the last working commit.
3. Start the incremental protocol: extract ONE function, write characterization tests, refactor, verify, commit.
4. Repeat. Each step takes 25 minutes and always leaves the code in a working state.

Key Insight

E

very anti-pattern shares a common thread: they arise from treating Claude as infinitely capable rather than as a powerful but bounded system. Context is finite. Attention degrades. Verification is essential. Working within these constraints—not ignoring them—is what separates productive Claude usage from frustrating Claude usage.

These are personal patterns. When you scale Claude to a team, new patterns emerge—and the solutions shift from personal discipline to organizational infrastructure. That is the subject of Part IV.

Try This

Audit your most active ML project:

1. Count lines in CLAUDE.md. Over 300? Flag for splitting.
2. Check your last 5 sessions: how many had 3+ correction rounds?
3. Is your project in an explicit development phase, or is it a permanent notebook prototype with no tests?
4. Identify the single anti-pattern most affecting your productivity. Apply the recovery guide above.

Tip

When debugging Claude itself, use `/debug` to inspect model, session, and configuration state. Use `/status` for version and account info.

Part IV.

Team & Enterprise

Ready for Level 4–5. Individual mastery is established; now the challenge is scaling it. This part covers automation pipelines, team governance, and enterprise deployment—the infrastructure that turns personal productivity into organizational capability.

13. Automation and Pipelines

Your team merges 20 PRs per week. Each needs type checking, test runs, and documentation updates. Manual review is not scaling. Claude can do this work—but not through interactive sessions.

Claude Code is not limited to interactive use. Its headless mode, fan-out capabilities, and the Claude Agent SDK enable production automation pipelines that run without human interaction. The extension mechanisms from Chapter 8—especially hooks—provide the building blocks. This chapter assembles them into pipelines.

13.1. Headless Mode

[Official] Use `claude -p` for non-interactive automation.[1]

`-p "prompt"`
: Single prompt, no interaction.

`--output-format json`
: Structured output for pipeline consumption.

`--allowedTools`
: Restrict to specific tools for safety.

`--model`
: Override model per invocation.

Official
`claude -p "prompt"` for non-interactive use. `--output-format json|stream-json|text`.
https://code.claude.com/docs/en/overview

13.1.1. Fan-Out Pattern

Before & After

Before:

```
Manual: open Claude, paste file, ask for type hints,
copy result, repeat 47 times.
```

After:

```bash
# Test on 3 files first
for f in $(find src/ -name "*.py" | head -3); do
  claude -p "Add type hints to $f" \
    --allowedTools Read,Edit
done

# Review results, then scale
find src/ -name "*.py" | while read f; do
  claude -p "Add type hints to $f" \
    --allowedTools Read,Edit \
    --output-format json >> results.jsonl
done
```

[Official] The `--allowedTools` flag scopes what Claude can do in unattended mode.[2] Use glob patterns for fine-grained control:

```
# Allow Read and Edit only (safest for code modifications)
```

[1] https://code.claude.com/docs/en/overview
[2] https://code.claude.com/docs/en/best-practices

```
--allowedTools "Read,Edit"

# Allow Bash but only for specific commands
--allowedTools 'Read,Edit,Bash(git commit *)'
```

Test Before Scaling

Always test your fan-out prompt on 2–3 files before running at scale. A prompt that works on `src/utils.py` may produce unexpected results on `src/models/network.py`. Review the first few outputs manually, then scale with confidence.

13.1.2. Permissions in Headless Mode

Headless Permissions

In headless mode, Claude still respects permission rules. For trusted CI/CD environments, use `--dangerously-skip-permissions` to bypass interactive prompts. This flag should **only** be used in environments you fully control (CI runners, Docker containers). Never expose it in user-facing scripts.

Tip

Combine `--allowedTools` with `--output-format json` for pipeline-friendly output. Parse results with `jq` for automated quality checks before committing.

13.2. Claude Agent SDK

Headless Mode vs. Agent SDK

Headless mode (`claude -p`): quick automation, scripting, CI/CD integration. Uses Claude Code's existing configuration. Best for: fan-out, batch operations, simple pipelines.

Agent SDK: programmatic control, custom tools, production deployment. Best for: custom applications, complex orchestration, SaaS integration.

Rule: Start with headless mode. Move to Agent SDK when you need custom tool definitions or programmatic control flow.

Official

Same agent loop as Claude Code, programmable in Python and TypeScript.
https://platform.claude.com/docs/en/agent-sdk/overview

[Official] The Agent SDK provides:[3]

- Filesystem-based configuration (CLAUDE.md per Chapter 9, skills, memory)
- Custom tool definitions and MCP server integration
- Streaming responses and structured output
- Production deployment with error handling and retry logic

13.3. Pipeline Design Patterns

13.3.1. Traffic-Light Dashboards

[Practitioner] Replace pass/fail with traffic-light thresholds (GREEN/YELLOW/RED). This provides nuance: 78% coverage is not "failing"—it is YELLOW, meaning "acceptable but track for improvement."

```
=== Project Health Dashboard ===
Test Coverage:    82% [GREEN]
Lint Warnings:     3  [YELLOW]
Documentation:   91% [GREEN]
Model Metrics:   AUC 0.84 [GREEN]
Overall:         HEALTHY (3 GREEN, 1 YELLOW)
```

[3] https://platform.claude.com/docs/en/agent-sdk/overview

13.3.2. Pipeline Directionality

Pipeline Direction

Never re-run upstream steps after editing downstream artifacts. Mark generated files with `# GENERATED --- do not edit manually`. This is anti-pattern #5 from Chapter 12.

Key Insight

A

utomation with Claude follows the same principle as automation without it: build the pipeline for the common case, handle exceptions manually. The difference is that Claude can handle a much wider "common case" than traditional scripts, reducing the exception rate dramatically.

Try This

1. Write a `claude -p` command that adds type hints to one file in your project.
2. Test it. Does the output compile? Do tests pass?
3. If yes: scale to 3 files. Review the results.
4. If no: refine the prompt. What did you need to specify that you assumed?

14. Team Patterns and Governance

You have been using Claude effectively for months. Your team lead asks: "Can we standardize this across our 8-person team?" The answer is yes—but individual mastery does not automatically translate to team productivity. Shared conventions require infrastructure.

14.1. Shared CLAUDE.md in Version Control

The project-level `CLAUDE.md` (structured per Chapter 9) and `.claude/settings.json` are committed to version control. Every team member gets the same project context, the same permissions, and the same hooks.

Official
Claude Code included with Team plan seats. Shared CLAUDE.md in VCS.
https://support.claude.com/en/articles/11845131-use-claude-code-with-your-team-or-enterprise-plan

[Practitioner] Three files form the team standard:

`CLAUDE.md`
: Project knowledge: build commands, architecture, conventions. Reviewed in PRs like any other code.

`.claude/settings.json`
: Team permissions: deny rules, required hooks. Committed to git.

`.claude/settings.local.json`
: Personal preferences: model choice, output style, personal API tokens. Auto-gitignored.

Why Separate Committed and Local Settings

If model choice is in `settings.json`, developers who need Opus for complex tasks must override team settings. If it is in `settings.local.json`, each developer chooses their own model without affecting others.

The principle: team standards (deny rules, hooks) are committed. Personal preferences (model, style) are local.

14.2. Managed Settings for IT

[Official] IT administrators can deploy `managed-settings.json` files that override all other settings.[1] This enables:

- Organization-wide deny rules (sensitive file access)
- Mandatory hooks (security scanning, compliance checks)
- Approved MCP server lists
- Model restrictions (e.g., Sonnet only for cost control)

Managed settings deploy to system-level paths and cannot be overridden by project or user settings (see Section 9.4 for the full 5-level precedence hierarchy). They are the IT equivalent of hooks—non-negotiable.

Hook Precedence and Debugging

When hooks are configured at multiple levels, they all run—managed, project, and local hooks execute for the same event. If hooks conflict (e.g., a managed hook requires formatting that a local hook undoes), managed takes priority.

[1] https://code.claude.com/docs/en/settings

To debug unexpected hook behavior:

- Check which hooks are active: review `settings.json` at each level
- Use `/hooks` to view the merged hook configuration
- Check hook exit codes: non-zero exit blocks the triggering action
- Test hooks in isolation before deploying to managed settings

14.3. Onboarding New Team Members

[Practitioner] Day 1 with Claude for a new team member:

1. Clone the repo. CLAUDE.md and settings load automatically.
2. Run `/init` to verify Claude understands the project. Compare output to the existing CLAUDE.md—they should align.
3. Create `.claude/settings.local.json` for personal preferences.
4. Run the team's `/check` command to verify the environment works.
5. Review the team's hook configuration: what runs on file write? On commit? Why?

The goal: a new developer should be productive with Claude within their first day, without reading a setup guide. The configuration in version control should be self-documenting.

14.4. Shared Extensions at Scale

In Chapter 8, you learned the extension hierarchy: commands, skills, hooks, and MCP servers. At team scale, these mechanisms need governance:

Shared skills
: Commit team skills to `.claude/skills/` in the repo. Quality dashboards, pre-release checklists, and onboarding workflows benefit the entire team. Use `context: fork` for expensive skills so they do not consume teammates' context budgets.

Managed MCP
: IT-approved MCP servers deployed via managed settings (Section 14.2). Prevents ad-hoc server installation while providing standardized access to team databases, ticketing systems, and monitoring tools.

Shared hook libraries
: Common hooks (auto-format, test gates, lint) maintained in a central repository and imported via the hub-and-spoke pattern (Section 9.1). Update once, all projects inherit.

[Practitioner] The extension hierarchy scales naturally: commands and skills in the repo for the team, hooks in `settings.json` for enforcement, managed MCP for IT-controlled integrations.

14.5. Code Review Patterns with Claude

[Practitioner] Two patterns for team code review with Claude (see also the Writer/Reviewer pattern in Section 10.4):

14.5.1. Pre-PR Self-Review

Before creating a PR, have Claude review your own changes:

```
Review the diff in `git diff main...HEAD` for:
1. Logic errors and bugs
2. Missing test coverage
3. Style violations per our CLAUDE.md
4. Security concerns
Report issues with file:line references and severity.
```

This catches obvious issues before reviewers see the code, reducing review cycles.

14.5.2. Reviewer-Assist

During review, use Claude to investigate specific concerns:

```
In PR #234, the author changed the feature
normalization strategy in
src/features/preprocessing.py.
What are the implications for models trained on
the old normalization? Check if any downstream
models depend on the old feature distributions.
```

Tip

The reviewer still makes the decision. Claude provides the analysis. AI augments review; it does not replace it.

14.6. Quick Reference

- Commit `CLAUDE.md` and `settings.json` for team standards
- Keep `settings.local.json` for personal preferences (auto-gitignored)
- Managed settings for IT policy enforcement (highest precedence)
- New team members productive on day 1 via committed configuration
- Pre-PR self-review catches issues before human reviewers see them

Try This

Draft a team `settings.json` with:

1. 3 deny rules protecting sensitive files (`.env*`, `secrets/`, credentials)
2. 1 ask rule for destructive operations (`git push`)
3. 1 PostToolUse hook (matching `Edit|Write`) for auto-formatting
4. 1 PreToolUse hook (matching `Bash`) gating commits on your test suite

Have a teammate review it. Does it match their expectations?

15. Enterprise Deployment

Your VP asks: "Is Claude secure enough for our regulated environment? What certifications does it have? What will it cost at 200 engineers?" This chapter is designed to be shared with decision-makers.

15.1. Security and Compliance

[Official] Anthropic holds industry-leading certifications:[1]

SOC 2 Type II
: Verified controls for security, availability, and confidentiality. Request the report via the Trust Portal.

ISO 27001:2022
: Information security management system.

ISO/IEC 42001:2023
: AI management system—one of the first AI companies certified.

FedRAMP High
: US government cloud security baseline (via Palantir FedStart).

HIPAA BAA
: Healthcare data protection agreements available on request.

Enterprise
Certifications verified Feb 2026. Trust portal:
https://trust.anthropic.com/
Certification details:
https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained

15.1.1. Identity and Access Management

[Official] Enterprise-grade IAM:[2]

- SSO via SAML 2.0 and OIDC
- SCIM provisioning for automated user lifecycle
- Role-based access control (RBAC)
- Audit logs for all interactions
- Compliance API and Analytics API for programmatic access

15.1.2. Data Protection

[Official] Key data protection features:[3]

- **Encryption**: TLS 1.2+ in transit, AES-256 at rest.
- **Zero Data Retention (ZDR)**: Available for API and Enterprise plans.
- **No training on enterprise data**: Enterprise and API data is not used for model training by default. Explicit opt-in required.

15.2. Deployment Scenario: 50-Person Rollout

[Practitioner] A realistic 4-week enterprise adoption timeline:

Enterprise
Enterprise/API data is never used for training by default. This is a contractual guarantee.
https://privacy.claude.com/en/articles/7996885-how-do-you-use-personal-data-in-model-training

[1] https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained

[2] https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained

[3] https://privacy.claude.com/en/articles/7996885-how-do-you-use-personal-data-in-model-training

Week	Phase	Activities
1	Pilot (5 developers)	Install Claude Code. Create shared CLAUDE.md. Establish deny rules. Identify 3 high-value workflows.
2	Expand (15 developers)	Deploy managed-settings.json. Add PreToolUse hooks. Create team commands. Measure baseline productivity.
3	Scale (50 developers)	SSO integration. Shared MCP configurations. Training sessions using this handbook. Monitor usage.
4	Optimize	Analyze usage patterns. Tune model selection for cost. Implement caching strategy. Set spending caps.

15.3. Cost Optimization at Scale

1. **Optimize CLAUDE.md for caching.** Stable tool definitions and system prompts cache automatically. Restructure to put stable content first.
2. **Use Batch API for bulk operations.** Code review, documentation generation, test scaffolding—all benefit from 50% batch discounts.
3. **Right-size models.** [Practitioner] Sonnet handles the majority of coding tasks. Opus for architecture and complex reasoning. Haiku for formatting and simple lookups.
4. **Set spending caps.** Admin controls prevent runaway costs from long-running agent sessions.

Cost
Prompt caching (up to 90%) + Batch API (50%) = up to 70%+ combined savings (workload-dependent).
https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Caching Beats Model Downgrading
The instinct to save money by using a cheaper model sacrifices quality across every interaction. Optimizing for cache hits preserves quality while reducing costs by a larger margin. A stable CLAUDE.md that caches well saves more than switching from Opus to Sonnet.

15.4. Enterprise Deployments at Scale

Organization	Scale	Use Case
Deloitte[4]	470K employees	Role-customized Claude "personas" for consulting
Accenture[5]	30K trained	CIO-focused ROI measurement
Snowflake[6]	12.6K customers	Text-to-SQL integration

15.5. Competitive Positioning

Claude vs. GPT by Task Profile
Choose Claude when: long-document analysis, agentic coding workflows, enterprise compliance requirements (FedRAMP High, ISO 42001), API-first development.
Choose GPT when: consumer-facing features, multimodal applications requiring image generation, existing GitHub Copilot integration.

Reality: most serious enterprise deployments use both, choosing by task profile. The question is not "which is better" but "which is better for this specific use case."

15.6. Government and Regulated Industries

[Practitioner] Government deployment options include:

FedRAMP High
: Available via Palantir FedStart integration.[7]

AWS GovCloud
: Cloud deployment for regulated requirements.[8]

Google Vertex AI
: Alternative cloud deployment for regulated workloads.[9]

15.7. When to Deploy: Decision Matrix

Need	Claude Code	Claude Chat	API / SDK
Interactive dev	Primary tool	Quick questions	—
Automation	Headless mode (Chapter 13)	—	Primary tool
Team standards	Settings + hooks (Chapter 14)	—	—
Compliance	Enterprise plan	Enterprise plan	API + ZDR
Custom apps	—	—	Agent SDK
Bulk operations	Fan-out	—	Batch API

Warning
Government program terms change frequently. Verify current offering details directly at https://claude.com/solutions/government before making procurement decisions.

Key Insight
E

nterprise deployment is not about buying seats—it is about building organizational capability. The technology is the easy part. The hard part is establishing shared conventions (CLAUDE.md standards), training teams on effective patterns (Chapter 14), and measuring ROI (usage analytics + quality metrics). See Chapter A for a framework to measure organizational adoption.

Try This
Estimate your team's monthly Claude cost:

1. Count developers × estimated sessions per day × average tokens per session.
2. Identify your top caching opportunity (what content is repeated in every session?).
3. Calculate: what would a 60% cache hit rate save monthly?
4. Present the estimate to your manager with the deployment scenario

[7] https://claude.com/solutions/government
[8] https://claude.com/solutions/government
[9] https://claude.com/solutions/government

> timeline from this chapter.

A. The Claude Maturity Model

This model provides concrete milestones for Claude Code adoption. Use it to assess where you are and what to invest in next. Each Part introduction in this handbook references the levels below.

A.1. Five Levels of Claude Adoption

Level	Name	Key Capabilities	Milestone
L1	Conversational	Default Claude, chat-style usage, no configuration	Create first CLAUDE.md (`/init`)
L2	Configured	CLAUDE.md, custom commands, permissions, basic hooks	5+ commands in a workflow chain
L3	Systematic	Hub-and-spoke, conditional rules, sessions, memory, plans	Hub repo with 3+ spoke projects
L4	Autonomous	Skills, MCP servers, subagents, parallel workflows, dashboards	Autonomous quality pipeline
L5	Enterprise	Agent SDK, agent teams, CI/CD, team-wide standards, SSO	Team-wide deployment with metrics

A.2. Upgrade Checklists

A.2.1. L1 → L2: From Conversational to Configured

- ☐ Create CLAUDE.md with build commands and code conventions
- ☐ Add `.claude/settings.json` with deny rules for secrets
- ☐ Create 3+ custom commands for common workflows
- ☐ Add PostToolUse hook (matching `Edit|Write`) for auto-formatting
- ☐ Add PreToolUse hook (matching `Bash`) gating commits on test suite

Reference: Chapter 2, Chapter 3

A.2.2. L2 → L3: From Configured to Systematic

- ☐ Create hub directory for shared patterns
- ☐ Convert 3+ projects to hub-and-spoke imports
- ☐ Add conditional rules (`paths:` frontmatter) for distinct modules
- ☐ Use sessions with named checkpoints (`/rename`)
- ☐ Maintain CURRENT_WORK.md for cross-session context
- ☐ Create first plan document for a large task
- ☐ Use thinking-partner techniques (hypothesis debugging, assumption surfacing)

Reference: Chapter 9, Chapter 6, Chapter 5

A.2.3. L3 → L4: From Systematic to Autonomous

☐ Create skills for repeated multi-step workflows
☐ Add MCP server for external tool access
☐ Define custom subagents for specialized tasks
☐ Implement writer/reviewer pattern for code review
☐ Build traffic-light quality dashboard
☐ Run parallel workflows with git worktrees
Reference: Chapter 8, Chapter 10

A.2.4. L4 → L5: From Autonomous to Enterprise

☐ Deploy managed-settings.json for organization-wide policy
☐ Implement headless mode in CI/CD pipeline
☐ Establish team CLAUDE.md standards committed to VCS
☐ Set up SSO and audit logging
☐ Build fan-out automation for bulk operations
☐ Track usage metrics and cost optimization
Reference: Chapter 13, Chapter 14, Chapter 15

A.3. Progression Strategy

[Practitioner] Promote when friction justifies it, not on a schedule.

1. **Stay at L1** until you find yourself repeating the same context to Claude across sessions.
2. **Move to L2** when CLAUDE.md and 3–5 commands would eliminate daily repetition.
3. **Move to L3** when 3+ projects share patterns or when plans would prevent wasted effort on large tasks.
4. **Move to L4** when multi-step workflows consume significant time or external tools would unlock new capabilities.
5. **Move to L5** when team adoption, compliance, or production automation requires enterprise infrastructure.

Premature Promotion

Jumping from L1 to L4 is a common mistake. Each level builds on the habits and infrastructure of the previous one. Skills without CLAUDE.md produce inconsistent results; MCP servers without permissions create security risks; agent teams without plan documents produce uncoordinated work.

A.4. Visual: Maturity Pyramid

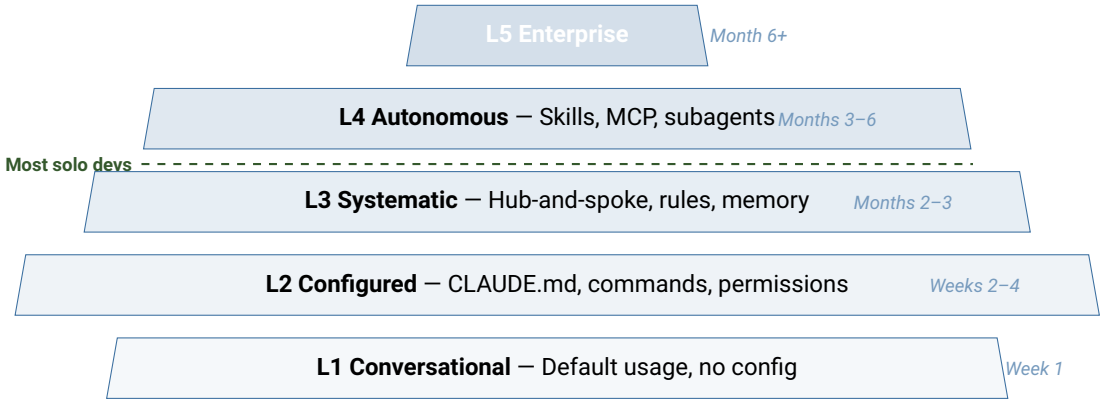

Figure A.1.: The maturity pyramid. Most solo developers stabilize at L3.

B. Quick-Start Templates

These templates are available as standalone files in the `templates/` directory of the repository. Copy them into your project and customize. Each template includes WHY comments explaining the purpose of each section.

B.1. Minimal CLAUDE.md (10 lines)

```
# Project Name

## Build & Verify
# WHY: Claude cannot guess non-standard build commands
- `pytest tests/` — run test suite
- `mypy src/` — type checking
- `ruff check src/` — lint

## Code Style
# WHY: Only include rules that differ from standard conventions
- Type hints on all function signatures
- Docstrings on all public functions

## Testing
# WHY: Verification criteria improve every interaction
- Run tests after every code change
- Never commit without passing lint + tests
```

B.2. Production CLAUDE.md (50 lines)

```
# Project Name

## Build & Verify
# WHY: Every command Claude needs to verify its own work
- `pytest tests/` — run test suite
- `mypy src/` — type checking
- `ruff check src/` — lint
- `nbstripout notebooks/` — strip notebook outputs

## Architecture
# WHY: Tells Claude WHERE things go, not just what they are
- src/features/ — feature engineering pipelines
- src/models/ — model training and evaluation
- src/data/ — data loading and validation
- notebooks/ — exploratory analysis
- data/raw/ — immutable source data
- data/processed/ — transformed artifacts
- tests/ — mirrors src/ structure

## Code Conventions
# WHY: Only rules Claude wouldn't infer from the codebase
- Type hints on all function signatures
- Functional style: prefer pure functions, immutable DataFrames
- Error handling: raise ValueError/TypeError, never swallow

## Testing Standards
# WHY: These lines improve every interaction (Ch 6)
- Run tests after every code change
- Never commit without passing lint + tests + type check
- Coverage target: 80% for src/ modules
- Every new function needs: happy path + error case + edge case
```

```
## ML Standards
- No data leakage: features must use only pre-split data
- Log all hyperparameters to MLflow
- Include reproducibility seed in all configs

## Git Conventions
- Conventional commits: feat/fix/refactor/test/docs
- PR description includes: Summary, Test Plan, Breaking Changes

## Security
# WHY: Defense in depth — CLAUDE.md + deny rules + hooks
- Never read .env files
- Never commit secrets, API keys, or credentials
```

B.3. Settings Template

```
{
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Write(**)", "Edit(**)",
      "Bash(pytest *)", "Bash(mypy *)", "Bash(ruff *)",
      "Bash(git status)", "Bash(git diff *)",
      "Bash(git log *)", "Bash(git add *)"
    ],
    "deny": [
      "Read(.env*)",
      "Read(secrets/**)",
      "Read(**/*.pem)",
      "Read(**/*.key)",
      "Read(**/credentials*)",
      "Bash(rm -rf /)",
      "Bash(sudo *)"
    ],
    "ask": [
      "Bash(git commit *)",
      "Bash(git push *)"
    ]
  }
}
```

Note: Permission rules support both glob syntax (`Bash(git push *)`, `Edit(/src/**/*.ts)`) and colon-style constraints (`WebFetch(domain:example.com)`). Evaluation order: deny first, then ask, then allow.

B.4. Hook Configuration

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/lint-on-write.sh",
        "description": "Auto-lint written files"
      }]
    }],
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
```

```
        "type": "command",
        "command": ".claude/hooks/pre-commit-gate.sh",
        "description": "Block commits with test failures or type errors"
      }]
    }]
  }
}
```

B.5. Skill Template

```
---
name: pipeline-check
description: Pre-merge pipeline readiness verification
allowed-tools: [Bash, Read, Glob, Grep]
---
# Pipeline Check
# WHY: Catches ML pipeline issues before they reach main
1. Verify all tests pass: `pytest tests/`
2. Check for uncommitted changes: `git status`
3. Verify no data leakage in feature code
4. Check model metrics logged to MLflow
5. Report readiness status (READY / NOT READY)
   with specific blockers if NOT READY
```

Note: Skills use `allowed-tools` (hyphenated).

B.6. Subagent Definition Template

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Review code changes for quality issues
tools: [Read, Grep, Glob]
---
# WHY: Isolated context prevents review bias from accumulated session context
Review the provided code changes for:
1. Logic errors and bugs
2. Missing error handling
3. Security vulnerabilities
4. Performance concerns
5. Style violations

Report findings with severity: Critical / High / Medium / Low.
Include file:line references for each finding.
```

Note: Subagents use `tools` (not `allowed-tools` or `allowed_tools`).

B.7. Rules File with Paths

```
---
paths: src/features/**/*.py
---
# Feature Engineering Standards
# WHY: These rules only load when editing feature code,
# keeping context clean when working on other parts of the project
- Pure functions: no side effects in transformers
- Validate input schema before transformation
- Type hints on all function signatures
- Docstrings on all public functions
- Run `pytest tests/features/` after changes
```

B.8. Custom Command Template

```
# .claude/commands/check.md
# WHY: Encodes a decision workflow, not just a shortcut
Run a comprehensive check of the project:

1. Run the test suite: `pytest tests/`
2. Run the linter: `ruff check src/`
3. Run the type checker: `mypy src/`
4. Report results as a summary table:
   | Check | Status | Details |
   |-------|--------|---------|
   | Tests | PASS/FAIL | X passed, Y failed |
   | Lint  | PASS/FAIL | N warnings |
   | Types | PASS/FAIL | N errors |
```

C. Command and Hook Reference

C.1. Built-In Commands

Command	Description
`/init`	Auto-generate CLAUDE.md from project structure
`/clear`	Clear context, start fresh
`/compact`	Summarize conversation, preserve key decisions
`/rewind`	Roll back to a previous checkpoint
`/context`	Visualize current context usage
`/rename "name"`	Give current session a descriptive name
`/hooks`	Configure lifecycle hooks interactively
`/mcp`	Manage MCP server connections
`/memory`	View and manage auto-memory
`/plugin`	Browse and install community plugins
`/sandbox`	Enable OS-level sandboxing
`/debug`	Show debug info: model, session, config, connection status
`/permissions`	View and modify permission rules
`/status`	Show version, model, and account info
`/help`	Show available commands

C.2. Session Management Flags

Flag	Description
`--continue`	Resume most recent session
`--resume`	Pick from list of previous sessions
`--from-pr 123`	Start session with PR context pre-loaded
`--worktree`	Create isolated git worktree for parallel work
`-p "prompt"`	Headless mode (non-interactive, single prompt)
`--output-format`	Output format: `json`, `stream-json`, `text`
`--allowedTools`	Restrict available tools (comma-separated)
`--model`	Override model: `opus`, `sonnet`, `haiku`
`--output-style`	Override output style

C.3. Keyboard Shortcuts

Shortcut	Action
`Shift+Tab`	Cycle permission modes (Default → Accept Edits → Plan)[1]
`Ctrl+G`	Open plan in external text editor
`Alt+T`	Toggle extended thinking
`Ctrl+C`	Cancel current generation
`Ctrl+B`	Background a running task
`Ctrl+O`	Toggle verbose mode (show hook output)
`Esc`	Stop Claude mid-action (context preserved)
`Esc + Esc`	Open rewind menu (restore conversation, code, or both)

C.4. Hook Events (17 Total)

Event	Matcher Target	Typical Use
`SessionStart`	How started	Load context, run health check
`UserPromptSubmit`	(no matcher)	Transform or validate input
`PreToolUse`	Tool name	Validate inputs, block operations
`PermissionRequest`	Tool name	Auto-allow/deny by rule
`PostToolUse`	Tool name	Auto-lint, log, trigger follow-up
`PostToolUseFailure`	Tool name	Error handling, retry logic
`Notification`	Notification type	Route to Slack, email
`SubagentStart`	Agent type	Initialize agent config
`SubagentStop`	Agent type	Process agent results
`Stop`	(no matcher)	Run cleanup tasks
`TeammateIdle`	(no matcher)	Notify when agent idles
`TaskCompleted`	(no matcher)	Post-task automation
`ConfigChange`	Config source	React to settings changes
`WorktreeCreate`	(no matcher)	Custom worktree setup
`WorktreeRemove`	(no matcher)	Custom worktree cleanup
`PreCompact`	Trigger type	Save state before compaction
`SessionEnd`	Why ended	Save state, update logs

Matchers are regex patterns. Tool events match on tool name (e.g., `Bash`, `Edit|Write`). Events marked "no matcher" always fire.

C.5. Hook Types

`command`

Runs a shell command (most common). Exit 0 = proceed, exit 2 = block.

prompt
: Single-turn LLM evaluation (Haiku by default).
Returns `{"ok": true/false}`.

agent
: Multi-turn subagent with tool access (up to 50 turns, 60s default timeout).

C.6. Settings Precedence (5 Levels)

1. **Managed** (`managed-settings.json`) — IT-deployed, highest priority
2. **CLI arguments** — temporary session overrides
3. **Local** (`.claude/settings.local.json`) — personal, auto-gitignored
4. **Project** (`.claude/settings.json`) — team standards, committed
5. **User** (`~/.claude/settings.json`) — global personal defaults

Higher levels override lower. Deny rules are absolute and cannot be overridden by lower levels.

D. Sources and Further Reading

All URLs verified as of February 2026.

Maintenance note: Pricing, model capabilities, compliance certifications, and government program terms change frequently. Verify these against primary sources before making purchasing or compliance decisions.

D.1. Claude Code Documentation

All Claude Code docs migrated from `docs.anthropic.com` to `code.claude.com` (301 redirects). URLs below use the canonical domain. Last verified February 2026.

Overview

`https://code.claude.com/docs/en/overview`

Best Practices

`https://code.claude.com/docs/en/best-practices`

Settings

`https://code.claude.com/docs/en/settings`

Hooks Reference

`https://code.claude.com/docs/en/hooks`

Hooks Guide

`https://code.claude.com/docs/en/hooks-guide`

Skills

`https://code.claude.com/docs/en/skills`

Sub-Agents

`https://code.claude.com/docs/en/sub-agents`

Memory

`https://code.claude.com/docs/en/memory`

MCP Integration

`https://code.claude.com/docs/en/mcp`

Agent Teams

`https://code.claude.com/docs/en/agent-teams`

Security

`https://code.claude.com/docs/en/security`

Settings Schema

`https://json.schemastore.org/claude-code-settings.json`

D.2. Prompt Engineering and API

Prompt Engineering Overview

`https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview`

Prompting Best Practices

`https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices`

Extended Thinking

https://platform.claude.com/docs/en/build-with-claude/extended-thinking

Prompt Caching

https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Batch Processing

https://platform.claude.com/docs/en/build-with-claude/batch-processing

Agent SDK Overview

https://platform.claude.com/docs/en/agent-sdk/overview

Advanced Tool Use

https://www.anthropic.com/engineering/advanced-tool-use

Pricing

https://platform.claude.com/docs/en/about-claude/pricing

D.3. Enterprise and Governance

Enterprise Plan

https://support.claude.com/en/articles/9797531-what-is-the-enterprise-plan

Team Claude Code

https://support.claude.com/en/articles/11845131-use-claude-code-with-your-team-or-enterprise-plan

Government Solutions

https://claude.com/solutions/government

Certifications

https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained

Data Retention

https://privacy.claude.com/en/articles/10023548-how-long-do-you-store-my-data

Training Data Use

https://privacy.claude.com/en/articles/7996885-how-do-you-use-personal-data-in-model-training

Trust Portal

https://trust.anthropic.com/

D.4. Partnership Announcements

Deloitte

https://www.anthropic.com/news/deloitte-anthropic-partnership

Accenture

https://www.anthropic.com/news/anthropic-accenture-partnership

Snowflake

https://www.anthropic.com/news/snowflake-anthropic-expanded-partnership

TELUS

https://www.anthropic.com/customers/telus

D.5. Safety and Research

Claude's Constitution (Jan 2026)

https://www.anthropic.com/news/claudes-constitution

Constitutional AI

https://www.anthropic.com/research/constitutional-ai-harmlessness-from-ai-feedback

Opus 4.6 Announcement

https://www.anthropic.com/news/claude-opus-4-6

D.6. Community Resources

Using CLAUDE.md Files

https://claude.com/blog/using-claude-md-files

How Anthropic Teams Use Claude Code

https://claude.com/blog/how-anthropic-teams-use-claude-code

Building Effective AI Agents

https://www.anthropic.com/engineering/building-effective-agents

Awesome Claude Code

https://github.com/hesreallyhim/awesome-claude-code

D.7. Further Reading

Claude Code Changelog

https://github.com/anthropics/claude-code/blob/main/CHANGELOG.md

Anthropic Engineering Blog

https://www.anthropic.com/engineering

Claude API Documentation

https://platform.claude.com/docs